



计算机组成原理
课程设计报告

专业班级：计算机类——19C243

学生姓名：孙泽龙

指导老师：樊媛媛

中国地质大学计算机学院

2023 年 6 月

目录

实验一、全相联 cache 设计	4
1.1 实验目的.....	4
1.2 主要任务.....	4
1.3 实验方案的总体设计.....	4
1.4 实验的实现过程.....	5
1.5 电路图及相关说明.....	6
1.5.1 Cache 基本结构.....	6
1.5.2 CAM 及其并发查找逻辑	9
1.5.3 读命中逻辑.....	10
1.5.4 读缺失逻辑.....	11
1.5.5 块写入逻辑.....	12
1.6 实验数据测试.....	12
实验二、4 路组相连 cache 设计	13
2.1 实验目的.....	13
2.2 主要任务.....	13
2.3 实验方案的总体设计.....	14
2.4 实验的实现过程.....	15
2.5 电路图及相关说明.....	16
2.5.1 Cache 基本结构.....	16
2.5.2 CAM 及其并发查找逻辑	17
2.5.3 读命中逻辑.....	17
2.5.4 读缺失逻辑.....	17
2.5.5 块写入逻辑.....	18
2.6 实验数据测试.....	20
实验三、2 路组相联 cache 设计	20
3.1 实验目的.....	20
3.2 主要任务.....	20
3.3 实验方案的总体设计.....	21
3.4 实验的实现过程.....	22
3.5 电路图及相关说明.....	22
3.5.1 Cache 存储部分基本结构.....	22
3.5.2 CAM 及其并发查找逻辑	23
3.5.3 读命中逻辑	23
3.5.4 读缺失逻辑.....	24
3.5.5 块写入逻辑.....	25
3.6 实验数据测试.....	25
实验四、单周期 MIPS CPU 设计	26
4.1 实验目的.....	26
4.2 主要任务.....	26
4.3 实验方案的总体设计.....	27
4.3.1 实验规划.....	27

4.3.2 总体设计.....	27
4.4 实验的实现过程（结合电路图说明）.....	28
4.4.1 指令解析.....	28
4.4.2 实现地址转移逻辑.....	29
4.4.3 单周期硬布线控制器.....	29
4.4.4 单周期 CPU 总体结构图.....	31
4.5 实验数据测试.....	32
4.5.1 头歌平台测试结果：.....	32
4.5.2 八条指令测试结果：.....	32
实验五、微程序地址转移逻辑设计.....	35
5.1 实验目的.....	35
5.2 主要任务.....	35
5.3 实验方案的总体设计.....	36
5.4 实验的实现过程（结合电路图说明）.....	36
5.4.1 微程序地址转移逻辑自动生成.....	36
5.4.2 使用组合电路分析功能连线.....	37
5.5 实验数据测试.....	37
实验六、MIPS 微程序 CPU 设计.....	38
6.1 实验目的.....	38
6.2 主要任务.....	38
6.2.1 完成指令集设计.....	38
6.2.2 完成电路逻辑.....	38
6.3 实验方案的总体设计.....	38
6.4 实验的实现过程（结合电路图说明）.....	39
6.4.1 从单周期到多周期.....	39
6.4.2 指令集的生成.....	39
6.4.3 多周期 CPU 总体结构图.....	41
6.5 电路图及其相关说明.....	41
6.5.1 取指令阶段数据通路.....	41
6.5.2 译码/取数阶段数据通路.....	42
6.5.3 其他指令及其指令执行.....	43
6.6 实验数据测试.....	44
6.6.1 头歌平台测试结果：.....	44
6.6.2 八条指令单独测试用例的测试结果.....	45
实验心得.....	49

实验一、全相联 cache 设计

1.1 实验目的

本实验旨在理解全相联映射 Cache 的基本原理和工作机制。全相联 Cache 作为 Cache 设计中的一种极端形式,允许主存中的任意块映射到 Cache 中的任意位置,从而最大化了 Cache 空间的利用率,但也带来了更高的硬件复杂性。同时,实验还致力于掌握全相联 Cache 的地址划分和查找方法,明确主存地址如何被解析为标记和块内偏移,以及这些信息在 Cache 查找过程中的作用。在此基础上,我进一步实现全相联 Cache 的并行比较机制,通过同时比较所有 Cache 行的标记,来高效判断数据是否命中,尽管这种并行比较需要大量的比较器硬件。最后,本实验也着重于熟悉替换算法在全相联 Cache 中的应用,特别是 LRU 等高效替换算法如何在 Cache 满时选择最佳的被替换块,以维持较高的命中率。

1.2 主要任务

为了达成上述实验目的,我首先设计了 16 位主存地址映射到 8 行全相联 Cache 的映射方案,明确了主存地址与 Cache 行之间的对应关系,这是整个 Cache 系统设计的基础。

接着,我实现了地址划分:将 16 位主存地址分为 14 位标记(Tag)和 2 位块内偏移(Offset),确保 Cache 能够正确解析和识别数据在主存中的位置及其在 Cache 块中的具体偏移。为了实现高效查找,我构建了 8 路并行比较器,这意味着可以同时比较所有 Cache 行的标记,从而在一次查找中快速判断数据是否命中,极大地提升了查找效率。

在 Cache 行满时,为了有效管理 Cache 空间,我实现了 LRU (Least Recently Used) 替换算法,确保 Cache 在需要替换旧数据块时,能够淘汰近期最少使用的块,以优化命中率。这一个任务又可以分为两个子任务:1. 使用 LRU 算法找到近期最少使用的块(具体实现时也需要考虑到如果存在还未使用的块,那么选择该块);2. 实现写入逻辑,在对应的块写入准备就绪的数据块。

最终,所有这些设计都被整合,我在 Logisim 中搭建了完整的全相联 Cache 电路,这包括了用于存储标记的 8×14 位寄存器组成的标记存储阵列,以及用于存储实际数据块的 8×32 位寄存器组成的数据存储阵列。并行比较的关键在于 8 个 14 位比较器组成的并行比较电路。此外,我还集成了优先编码器来处理可能的匹配情况(尽管全相联 Cache 理论上只有唯一匹配),并完善了 LRU 替换逻辑,使其能够准确跟踪和更新 Cache 行的使用情况,从而实现正确的替换决策。

1.3 实验方案的总体设计

为了实现全相联 cache 高速缓冲存储器,我设计了下面的五个模块,分别对应了上述提到的具体任务:

- 构建 Cache 槽,实质是相连存储器。Cache 负责存储数据块和保存辅助

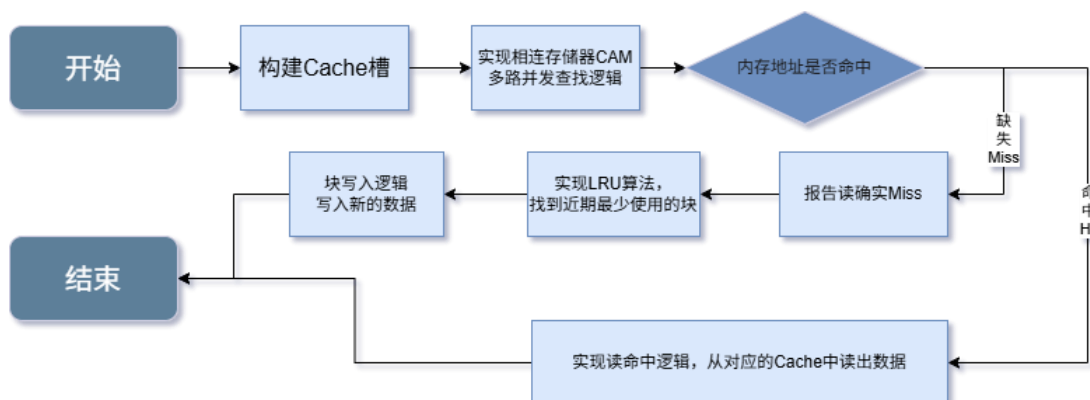
信息，如写使能信号、Valid 位、命中信号 Li 等，并提供控制读取和辅助信息的接口。

- 实现相连存储器 CAM 的多路并发比较逻辑。比较地址中的 Tag 和每一个 Cache 槽的有效位信息和 Tag_i 。
- 实现读命中逻辑。当内存地址命中 Hit 时，直接从 Cache 中读出。
- 实现读缺失逻辑。当内存地址没有命中 Miss 时，需要报告读缺失 Miss；同时还需要实现 LRU 算法，找到符合条件的槽，并交给块写入逻辑实现写操作。
- 实现块写入逻辑。收到 Miss 信号时，要在对应的存储块中写入新的数据块。

通过这五个模块的紧密协作，全相联 Cache 能够实现高效的数据存储、查找、命中处理、缺失处理及数据更新，从而实现全相联 Cache 的主要功能。

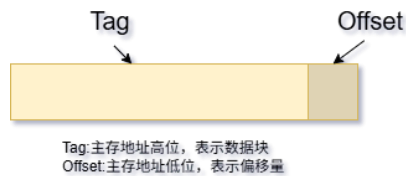
1.4 实验的实现过程

本实验严格遵循 Cache 的实际工作流程进行设计与实现，各功能模块的开发顺序与 Cache 处理访存请求的时序逻辑完全吻合。从基础的存储结构构建开始，逐步实现了地址解析、并行查找、命中判断等核心功能，最后完成缺失处理和替换算法，整个实现过程真实模拟了 Cache 硬件在实际运行时的完整工作序列。这样的实现顺序不仅确保了各功能模块之间的逻辑连贯性，也使我在开发过程中能够更加直观地理解 Cache 系统从接收到访存请求到最终完成数据访问的完整处理流程。每个阶段的实现都严格对应 Cache 实际工作中的特定环节，从而保证了最终实现的 Cache 系统能够准确反映真实硬件的行为特征。下图所示为 Cache 系统的工作流程示意图，该图完整呈现了 Cache 处理访存请求的标准工作顺序。该流程图不仅清晰展示了 Cache 的核心工作流程，也与本实验的实现顺序保持高度一致，为理解 Cache 的工作原理提供了直观的参考依据。



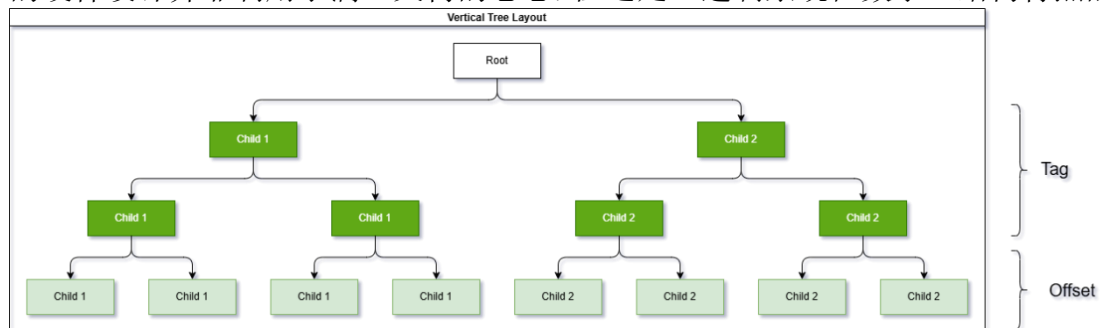
度一致，为理解 Cache 的工作原理提供了直观的参考依据。

整个 Cache 系统的工作流程将这些模块有机整合：接收访存请求后首先解析地址，然后并行查找所有 Cache 行并判断命中情况。命中时直接执行读写操作，未命中时则启动缺失处理流程，包括选择替换行、从主存读取数据、更新 Cache 内容等步骤，最终完成访存请求。通过这一系列精心设计的硬件逻辑，我实现了一个高效可靠的全相连 Cache 系统。



Cache 的本质仍然是一个存储系统，最核心的功能是根据给出的地址找到对应的数据。每个字节地址都是一个 16 位数据，0-1 位是 offset，表示在 4 字节的数据块中的偏移量；2-15 位是 tag，是在内存中的数据块位置。如果考虑一个无穷大且速度无穷快的存储系统，也就是 cache 的理想情况，那么每给出一个内存地址，都会先根据 tag 找到对应的内存块，再根据 offset 找到具体的字节地址。

一种形象的理解是：一个地址表示的是一个满二叉树中叶节点位置，地址高位决定大方向，地址低位则是这棵满二叉树上末端的几支中的地址。虽然 cache 的硬件设计并非利用了满二叉树的思想，但这是二进制系统在数学上结构特点的



通用形象表示。

我将全连接 Cache 的实现过程分为下面五步，以此实现：

1. 构建存储区域和接口：实现八个 cache 槽，用来存储，读出和写入数据，并提供相应的接口。
2. 实现并发查找逻辑，原理依据是相联存储器。
3. 实现读命中逻辑，完成电路，直接将读到的数据输出。
4. 实现读缺失逻辑，利用算法找到最合适写入或替换成新数据（从主存中引入）的 cache 槽，报告给 块写入逻辑。
5. 实现块写入逻辑，将主存中的数据块转移到合适的 cache 槽中（由读缺失逻辑提供）。

每一步的详细实现会在`1.5 电路图及相关说明`中结合电路图说明。

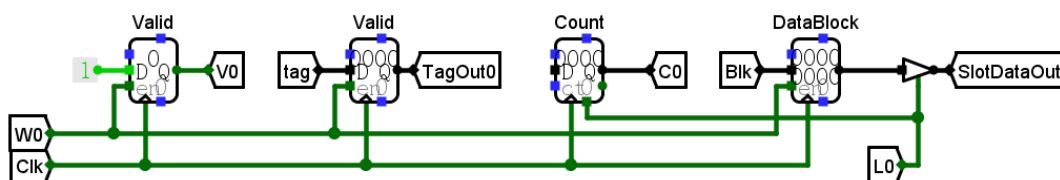
1.5 电路图及相关说明

1.5.1 Cache 基本结构

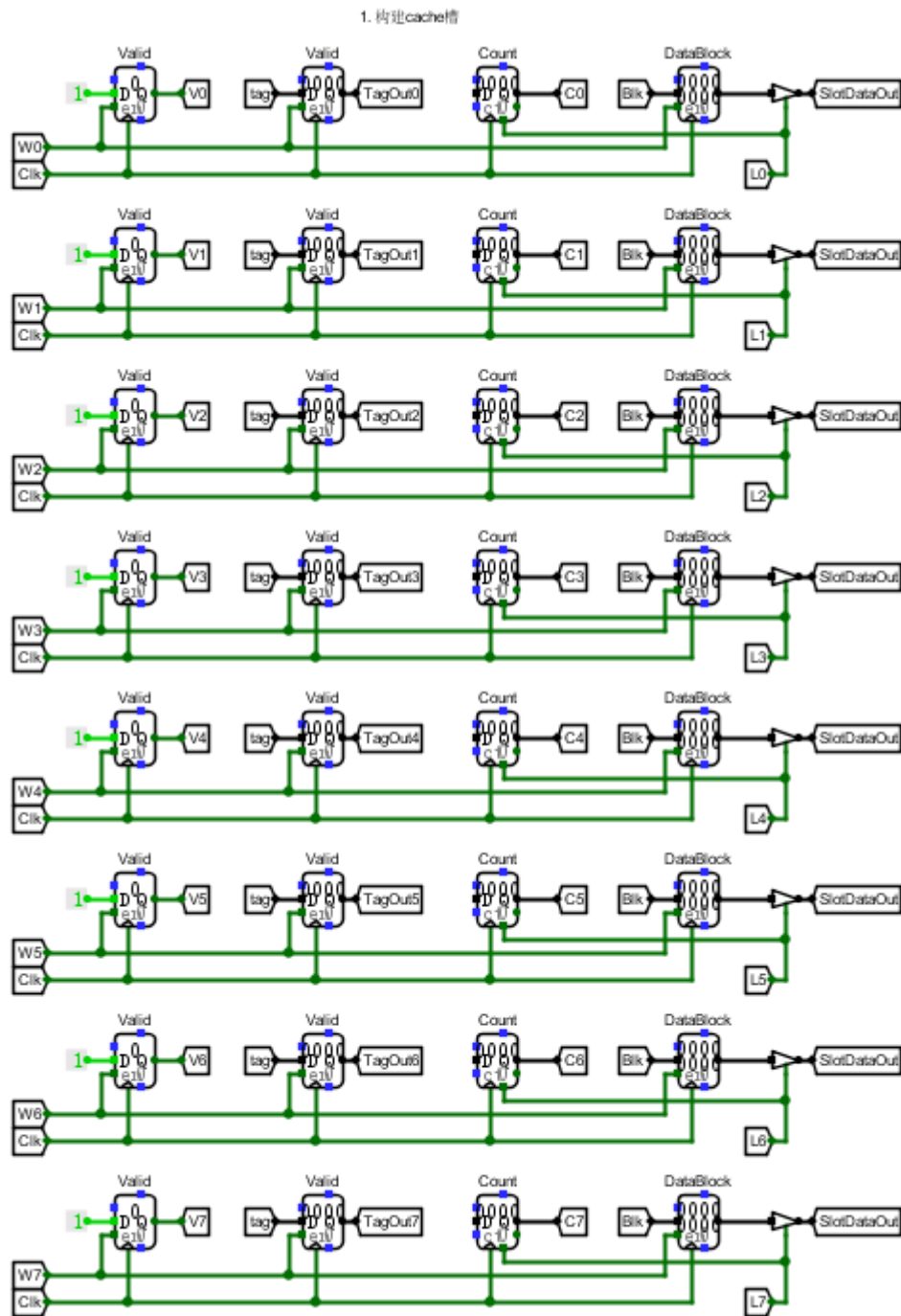
首先进行了 Cache 基本结构的构建，我设计了由多个 Cache 行组成的存储阵列，每个 Cache 行包含有效位、标签位和数据块三个主要部分，以及用于辅助 LRU 算法实现的最近访问时长计数器。采用全相连映射方式意味着主存中的任何数据块可以被放置到 Cache 的任何位置，这要求实现能够并行访问所有 Cache 行的硬件结构。初始化时将所有 Cache 行的有效位清零，表示初始状态下 Cache

为空。其要点如下：

1. 有效位：标记当前 Cache 行是否存储有效数据。
2. Tag：存储主存地址的高位部分，提供输入输出通道，用于写入新的 Tag 和输出 Tag 以供项链存储器比较。
3. Count：支持 LRU 替换策略，记录 Cache 行的“年龄”。命中时：命中行的计数器清零，其他行计数器递增。缺失时：选择计数值最大的行（最久未使用）替换。这些操作逻辑都提供接口，在其他电路逻辑判断之后可以直接调用接口实现。
4. DataBlock 数据块：存储从主存加载的数据块内容。SlotDataOut 为数据输出端口，位宽等于数据块大小。读命中时：直接输出对应数据块。
5. 写使能信号 Wi：是实现写入逻辑的接口，允许写入时，置 Wi 位为 1，则同步更新有效位，Tag 和数据块内的信息。
6. 命中信号 Li：标记当前访问是否命中该行，命中时则置 Li 为 1，便可以执行对应的操作，包括 Count 清零和利用三态门选择对应的数据块输出。

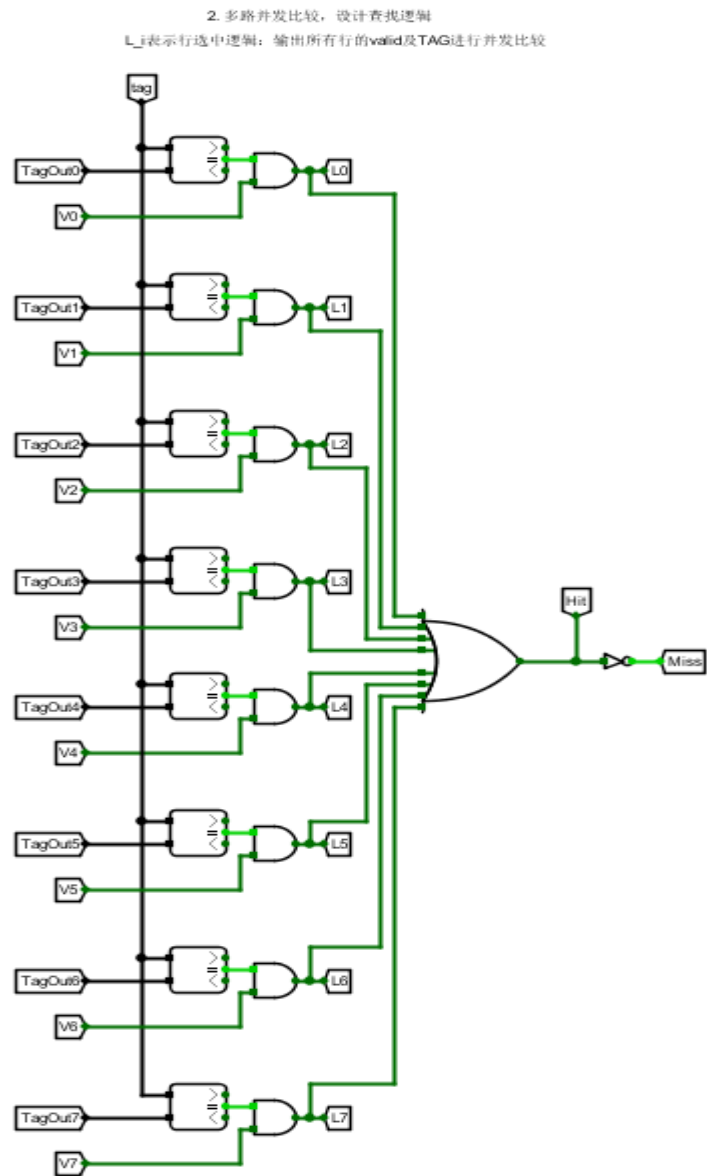


将八个这样的 cache 组合起来，并完善合适的标号，就得到完整的 cache 存储结



构和其接口。

1.5.2 CAM 及其并发查找逻辑



Cache 的并发比较逻辑本质上就是实现一个相联存储器 CAM，下图展示了相

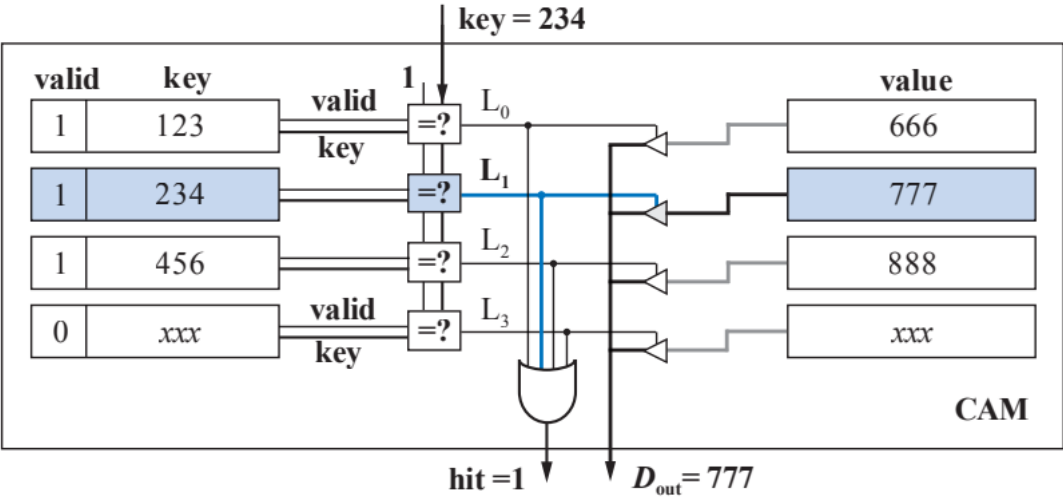


图 4.35 CAM 的基本原理

联存储器的工作原理。转换成电路逻辑就是：分别判断 tag 值是否等于目标地址，valid 是否等于 1。通过为每个 Cache 行配备独立的比较器，系统能够在时钟周期内同时比较所有行的标签与访存地址的标签部分。无论结果如何，都将信号传导到三态门，从而决定是否允许将 data 输出成 D_{out} ，同时对每一行的比较结果做或运算，得到 Hit 信号。Hit 信号为 1 时则判定命中，为了方便后续处理，我还对 Hit 信号做非运算，得到 Miss 信号。

1.5.3 读命中逻辑

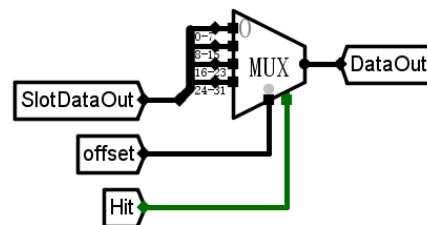
读命中逻辑要解决的问题有三个：

1. 当且仅当 Hit 命中时才需要调用读命中逻辑；
2. 根据 Offset 从数据块中选择正确的字节。
3. 辅助 LRU 算法的实现，将计数器 Count 置零。

前两点可以通过分线器+多路选择器实现。先将数据块分线成 4×8 段共 4 段，利用多路选择器根据 Offset 选择合适的字节。使用带有使能端的多路选择器，与 Hit 相连，从而利用 Hit 对电路进行控制。

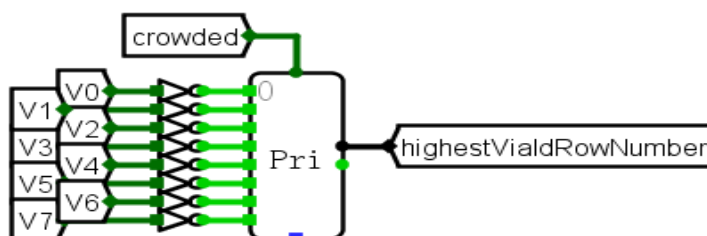
第三点利用 Count 计数器的清零接口就可以实现，只需将匹配成功或者 Hit 信号传给计算器的清零接口就可以。这里选择直接传递 Li。

电路实现如右图所示。



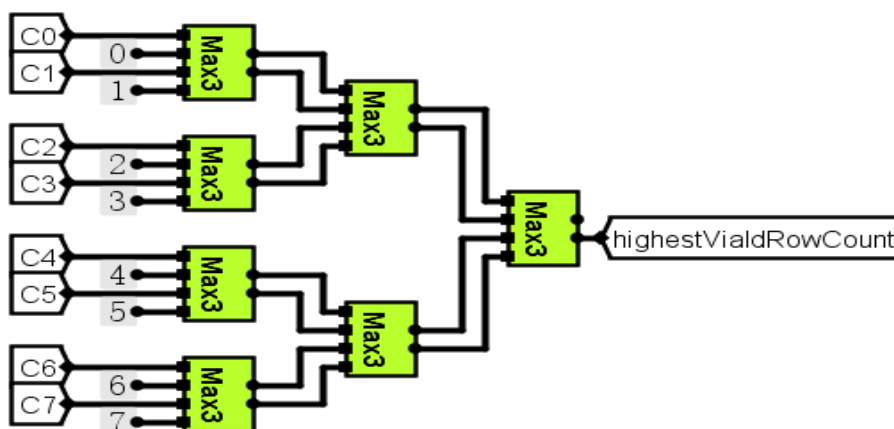
1.5.4 读缺失逻辑

4. 读缺失逻辑



优先解码器：输出最高1位的编号，八位1000，0000则输出111，不允许无输入1。

类似 菊花链



当处理器尝试从缓存中读取数据，但未找到所需数据（即发生读缺失）时，系统会生成一个缺失信号。这个信号会触发缺失处理流程，目的是从主存中加载所需的数据块到缓存中。

在缺失处理流程中，首先需要确定哪个缓存行将被用来存储从主存加载的数据块。这涉及到替换算法的实现，以决定哪个现有的缓存行应该被新加载的数据块替换。

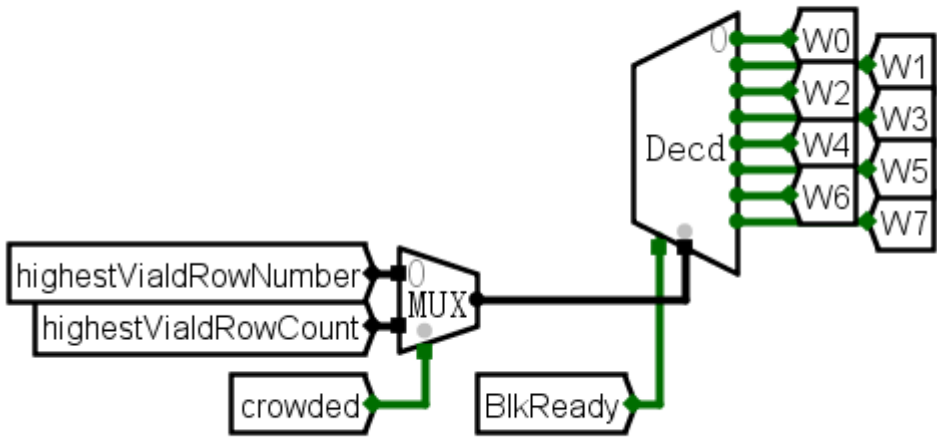
我使用了优先解码器来实现最基本的替换逻辑，即还存在未占用的 cache 槽。这个优先解码器类似于菊花链结构，用于输出最高 1 位的编号。具体来说：它接收多个输入（V0 到 V7），并输出最高有效位（Pri）。这个输出用于确定哪个缓存行应该被替换。例如，如果输入中 V7 是最高有效位，则输出为 111，表示第 7 行应该被替换。

当所有 cache 都被占用时，首先优先阶码器会报告拥挤信号 Crowded（这是用来启动块写入逻辑），然后根据 LRU 算法，选择最近最少使用的 cache 槽来进行替换，即替换掉选择计数器值最大的槽中的数据。

在这一步中，只需要选择出需要替换的行即可，具体的替换操作则由块写入逻辑完成。

1.5.5 块写入逻辑

5. 块写入逻辑



块写入逻辑负责处理缺失时的数据调入操作。

首先，读缺失逻辑电路会产生表示是否拥挤的 Crowded 信号，我使用多路选择器根据 Crowded 信号选择使用最优先空行还是 LRU 算法决策出的原本有数据的行。这样就会产生一个行号。再利用对这个行号做一次解码，便能产生对应行的写使能信号。

注意，本实验还提供了 BlkReady 信号（表明数据是否可以写入），因此我对 Decd 增加一个使能端口，使得只有数据准备就绪时才会进行写入操作。

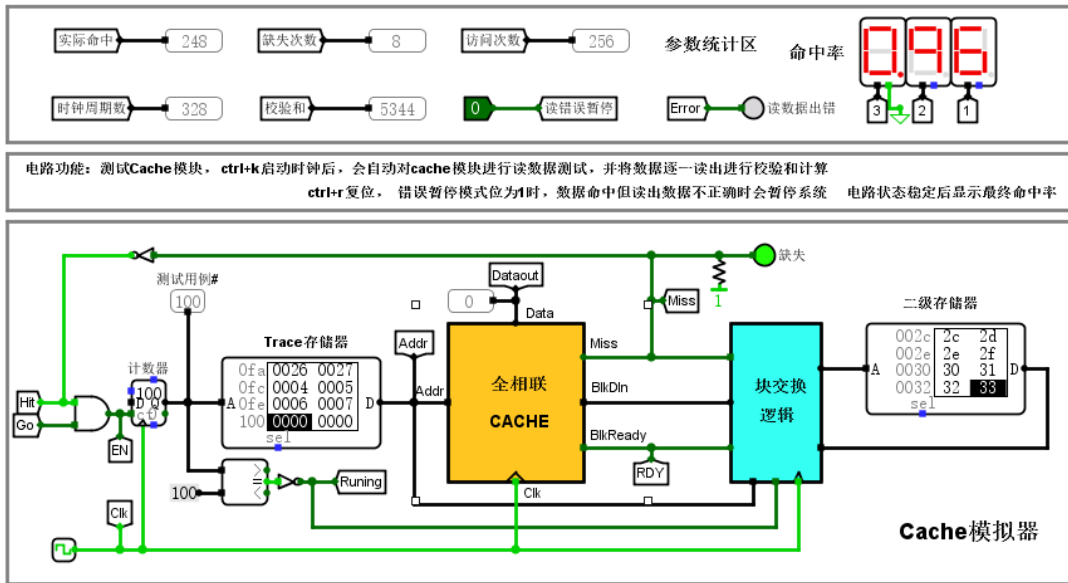
最后，对应行的写使能信号通过 Cache 槽的接口实现对特定 Cache 槽数据的替换。

1.6 实验数据测试

EduCoder 平台通过的截图：

4	全相联cache设计	2025-06-27 15:22	24598	1	2025-06-27 22:01	6时 39分 35秒	否	600/600	20.00/20	20.00
---	------------	------------------	-------	---	------------------	------------	---	---------	----------	-------

Cache 自动测试电路的运行截图如下：



如果设计了不同的映射方式，可以将该电路中cache模块替换为对应Cache模块即可！
默认trace是顺序访问，所以测试系统中判断读错误的逻辑是地址和数据相等，如果更换了trace，这个逻辑不正确

实验二、4 路组相连 cache 设计

2.1 实验目的

本实验旨在掌握 4 路组相联 Cache 的基本原理与设计方法。组相联 Cache 通过折中全相联与直接映射的优点，既降低了硬件复杂度，又保持了较高的命中率。实验需实现以下目标：

1. 地址划分与映射机制：理解主存地址如何划分为标记（Tag）、组索引（Index）和块内偏移（Offset），明确数据块在 Cache 中的组内灵活映射特性。
2. 并行比较与命中判断：设计 4 路并行的组内比较逻辑，通过同时比较组内所有行的标记位，快速判断数据是否命中。
3. 替换算法实现：在组内冲突时，应用 LRU（Least Recently Used）算法选择替换目标，优化 Cache 利用率。

2.2 主要任务

为实现上述目标，除了构建基础 Cache 槽并提供读出写入等信息接口以外，实验分还需要完成以下核心任务：

1. 地址划分设计：将 16 位主存地址划分为 13 位标记（Tag）、1 位组索引（Index）和 2 位块内偏移（Offset），确保组索引能定位 4 个候选 Cache 行。
2. 组内并行比较：依照 CAM 设计并行比较逻辑电路，实现组内标记的并行

匹配，生成命中信号（Hit）和行选择信号（Line Select）。

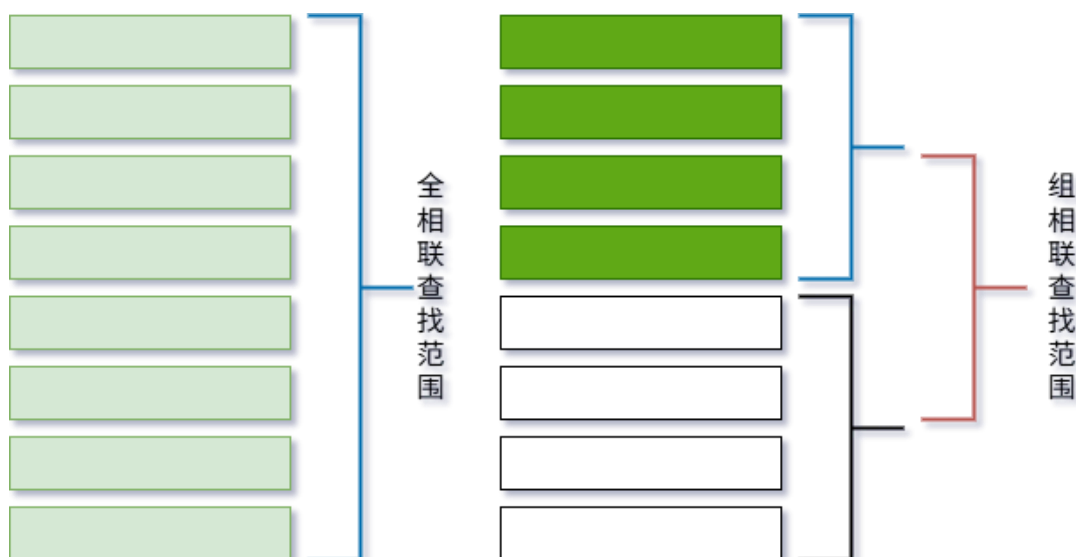
3. **LRU 替换策略**：为每组维护一个 Count 计数器，记录行访问顺序，替换时选择组内最久未使用的行。
4. **数据读写整合**：在 Logisim 中构建由 2 组×4 行组成的存储阵列，集成标记存储、数据块存储、比较电路及 LRU 更新逻辑，形成完整 Cache 系统。

2.3 实验方案的总体设计

为了实现 4 路组相联 cache 高速缓冲存储器，我设计了下面的五个模块，分别对应了上述提到的具体任务：

- 构建 Cache 槽，实质是相连存储器。Cache 负责存储数据块和保存辅助信息，如写使能信号、Valid 位、命中信号 Li 等，并提供控制读取和辅助信息的接口。
- 实现相连存储器 CAM 的多路并发比较逻辑。比较地址中的 Tag 和每一个 Cache 槽的有效位信息和 Tag_i 。
- 实现读命中逻辑。当内存地址命中 Hit 时，直接从 Cache 中读出。
- 实现读缺失逻辑。当内存地址没有命中 Miss 时，需要报告读缺失 Miss；同时还需要实现 LRU 算法，找到符合条件的槽，并交给块写入逻辑实现写操作。
- 实现块写入逻辑。收到 Miss 信号时，要在对应的存储块中写入新的数据块。

通过这五个模块的紧密协作，4 路组相联 Cache 能够实现高效的数据存储、查找、命中处理、缺失处理及数据更新，从而实现 4 路组相联 Cache 的主要功能。

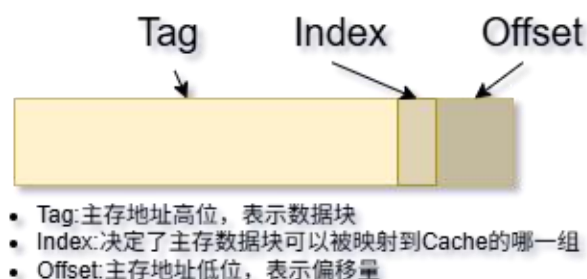


4 路组相联和全相联的区别（仅考虑这里的两组 4 路组相连）在于，在判断读缺失逻辑时，只需要考虑某一组内的替换位置进而在块写入时也只需要考虑在某一组内的写入位置。这样的设计使得组相联相比全相联在少量比较器下仍保持较高命中率，适合现代 CPU 设计。

2.4 实验的实现过程

本实验严格遵循**组相联 Cache**的实际工作流程进行设计与实现，各功能模块的开发顺序与 Cache 处理访存请求的时序逻辑完全匹配。从**存储结构构建**开始，逐步完成**地址解析**、**组选择**、**并行比较**、**命中判断**、**缺失处理**及**替换策略**等核心功能，整个实现过程模拟了真实硬件的工作机制。这样的实现顺序不仅保证了模块间的逻辑一致性，也使我能够直观理解 Cache 从接收访存请求到完成数据访问的完整流程。每个阶段的实现均对应 Cache 实际工作的关键环节，确保最终系统能准确反映硬件行为。实现流程与全相联 Cache 基本一致，可参照 1.4 中的工作流程图，它为理解组相联映射原理提供了清晰的参考。Cache 的核心功能仍是**通过地址快速定位数据**。

在 4 路组相联设计中，每个 16 位地址被划分为：



4 路组相联 Cache 的实现步骤

1. 构建存储结构与接口：
 - 实现 2 组×4 行的 Cache 槽，每行包含 Valid 位、Tag、DataBlock（4 字节）及 LRU 计数器。
 - 提供读写接口（如写使能信号、数据输出端口）。
2. 实现组选择与并行比较逻辑：
 - 通过 Index（1 位）选择目标组，激活组内 4 行的 Tag 比较器。
 - 4 路并行比较输入 Tag 与各行 Tag，生成命中信号（Hit）及行选择信号（Line[1:0]）。
3. 实现读命中逻辑：
 - 命中时，根据 Offset（2 位）从 DataBlock 中选择对应字节。
 - 更新 LRU 状态：命中行的计数器清零，其他行递增。
4. 实现读缺失与替换逻辑：
 - 若未命中，通过 LRU 策略选择组内最久未使用的行（计数器最大值）。
 - 生成替换行号，触发主存数据加载请求。
5. 实现块写入逻辑：
 - 从主存读取数据块后，根据替换行号更新目标行的 Tag、DataBlock 及 Valid 位。
 - 重置该行的 LRU 计数器，标记为最新使用。

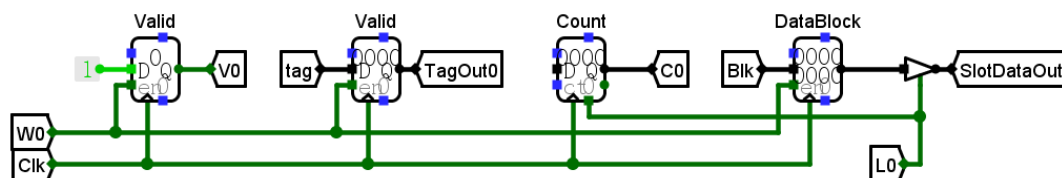
通过这种设计，4 路组相联 Cache 在**硬件成本**、**命中率**和**访问速度**之间取得了最优平衡，成为现代 CPU Cache 的典型方案。

每一步的详细实现会在`2.5 电路图及相关说明`中结合电路图说明。

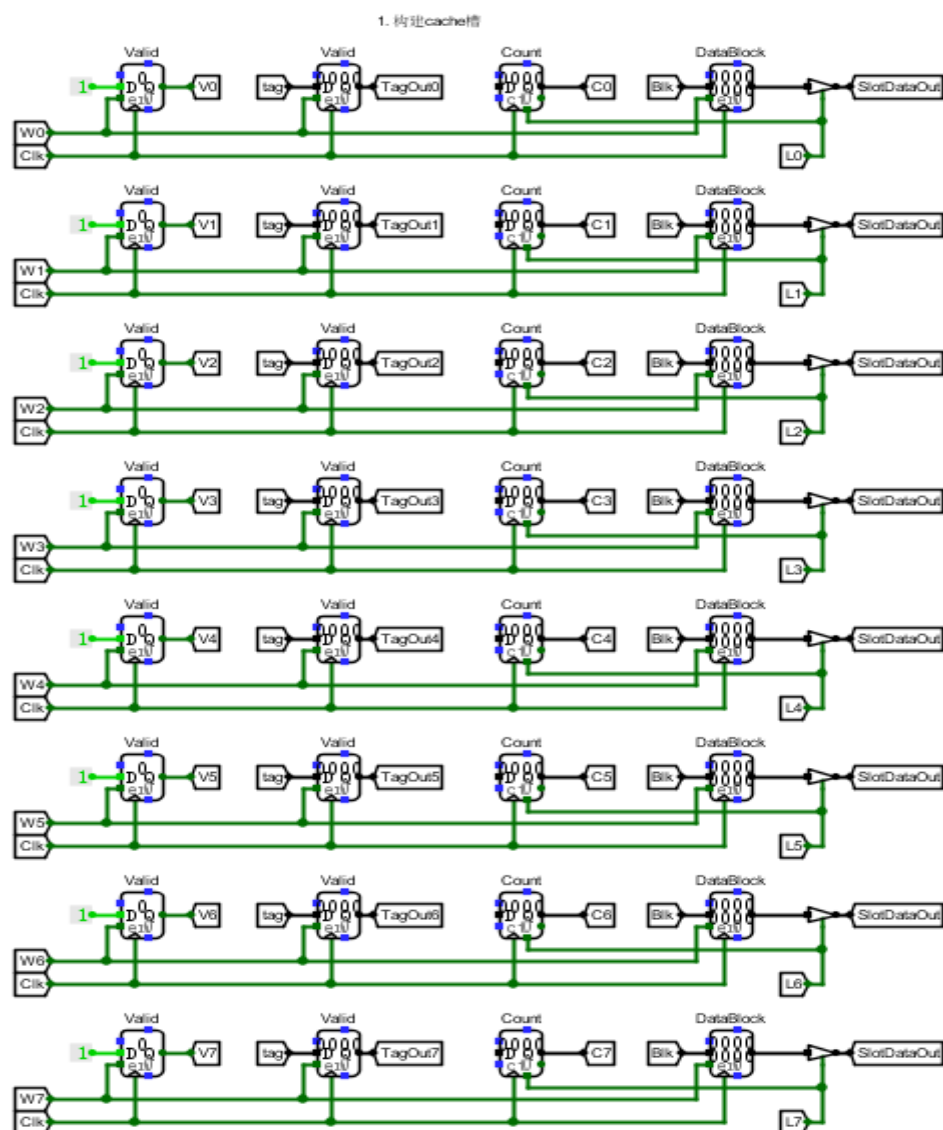
2.5 电路图及相关说明

2.5.1 Cache 基本结构

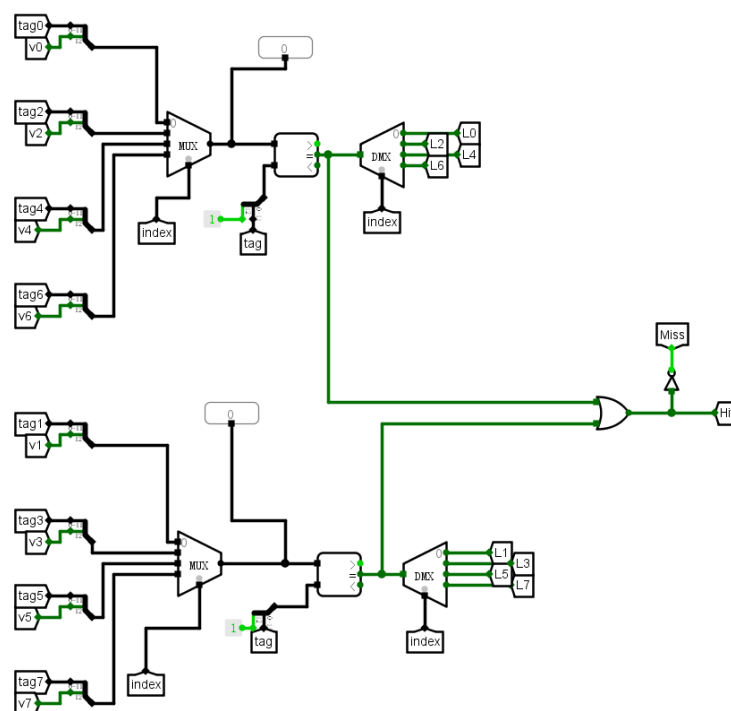
这部分与全相联 Cache 槽的设计完全一致。



将八个这样的 cache 组合起来，并完善合适的标号，就得到完整的 cache 存储结构和其接口。



2.5.2 CAM 及其并发查找逻辑



在 4 路组相联 Cache 的 CAM 及其并发查找逻辑设计中，核心思想是通过组索引（Index）选择目标组后，在组内实现并行标记比较。当 16 位主存地址输入时，系统首先解析出 1 位 Index 字段，通过 1-2 译码器选中对应的 Cache 组，同时提取 13 位 Tag 用于后续比对。该组内的 4 个 Cache 行会同时输出各自的有效位（Valid）和标记位（Tag_i），这些信号与输入 Tag 通过 4 个独立的 13 位比较器进行并行比较。比较器的输出会与对应行的 Valid 位进行逻辑与运算，只有当 Valid=1 且 Tag 匹配时，才会生成该行的命中信号（Hit_i）。所有命中信号通过或门汇总为最终的全局命中信号（Hit），同时通过优先编码器确定具体的命中行号（Line Select）。这种设计只需 4 个比较器即可完成组内全并行查找，既保留了全相联的灵活性，又大幅降低了硬件开销。

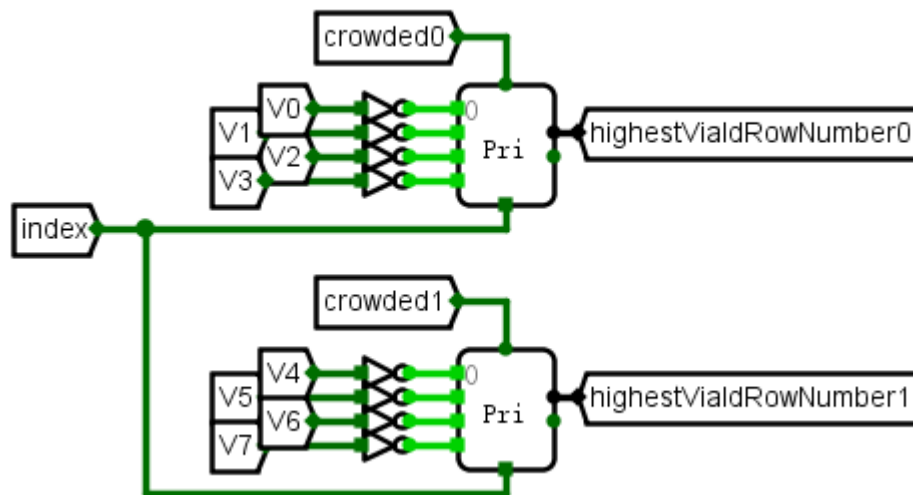
值得注意的是，Index 的选择逻辑使得不同组的比较器可以分时复用，进一步优化了电路规模，而有效位的引入确保了无效数据不会产生误命中。整个并发查找过程在一个时钟周期内完成，包括组选择、标记比较和命中判断三个关键阶段，完美体现了组相联 Cache 在访问速度和硬件成本间的平衡优势。

2.5.3 读命中逻辑

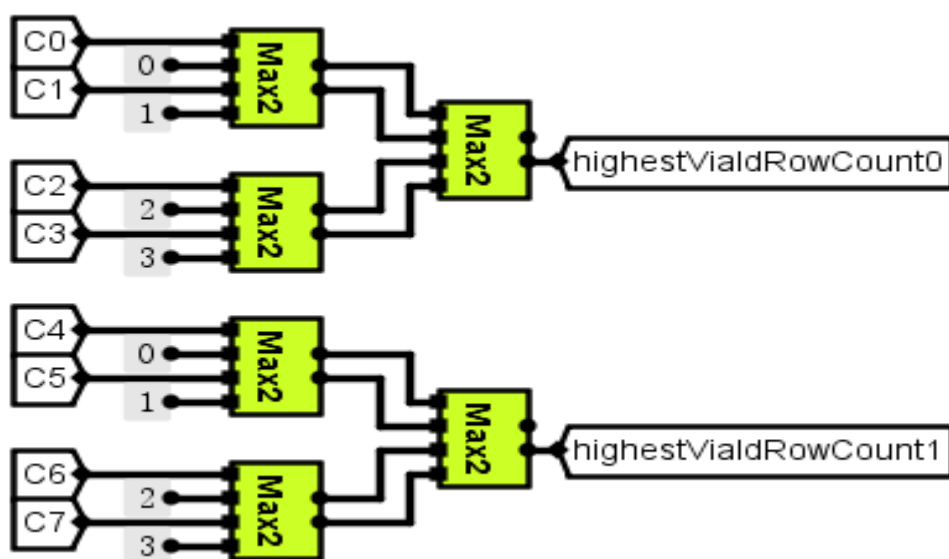
这部分的电路实现和全相联完全一致，也就是说，命中时直接从数据块中按照 Offset 选择对应字节地址的数据即可。

2.5.4 读缺失逻辑

首先我们可以明确一点，在每一个组内，相当于实现了一个全相联。在本实验这个具体的任务中，一共两组每组 4 行，我只需要对每一组做一个全相连同样的实现就可以。唯一的区别在于，可以用 Index 连接到优先选择器的使能端（不连接也不影响，只要在后面补充有关的选择逻辑），保证只在对应的组中选择。



LRU 算法的实现部分也与全相联一致，只需要在每一组内分别递归实现即可。



2.5.5 块写入逻辑

在 4 路组相联 Cache 的多组协同工作环境下（本设计共有两组，每组 4 路），块写入逻辑需要建立高效的组间协调机制。地址结构采用 13 位 Tag + 1 位 Index + 2 位 Offset 的划分方式。

当发生读缺失（Read Miss）并需要进行块写入时，系统首先通过地址中的 1 位 Index 字段确定目标组。这个 Index 字段将访存请求精准路由到对应的组处理单元。如图所示，有两个独立的组（表示为组 0 和组 1）处理单元。每个组都独立维护自己的 LRU 状态机（由 highestVialdRowNumber 和 highestVialdRowCount 等信号隐含，用于选择替换的行）和空闲行检测电路（由

Crowded 信号表示拥挤状态），但所有组共享统一的数据总线和控制信号，确保主存数据可以高效分发到各个组。

多组环境下，引入了分布式仲裁和协同写入机制：

1. 组内行选择与多路复用：

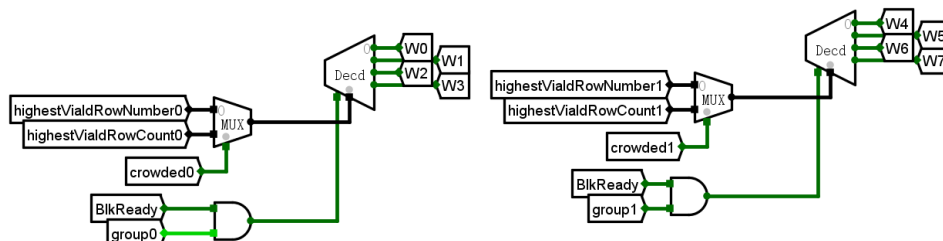
每个组（例如组 0）内部，highestVialdRowNumber0 和 highestVialdRowCount0 信号输入到一个多路选择器（MUX）。这些信号可能用于选择一个最优的替换行（例如基于 LRU 策略），或者选择一个空闲行。crowded0 信号也作为 MUX 的一个输入，它指示了当前组是否已经拥满（即没有空闲行）。MUX 的输出信号与 BlkReady0 和 group0 信号的 AND 操作结果共同驱动一个译码器（Decd）。译码器（Decd）根据输入信号，生成具体的写入控制信号（例如 W0, W1, W2, W3 对于组 0；W4, W5, W6, W7 对于组 1），这些信号用于选择并使能目标行进行数据写入。图中可以看出，每个译码器有 4 个输出，对应一个组内的 4 个路。

2. 分布式仲裁与块就绪信号：

crowded0 和 crowded1 信号指示了各自组的拥挤状态。这些状态信息会被汇总（图中未直接画出汇总逻辑，但其概念是存在的）到中央控制单元或通过分布式逻辑进行处理。控制单元可能采用优先级编码器（或更简单的逻辑）处理并发写入请求，确保在多个组同时需要更新时，能够按照预设的优先级顺序依次处理。BlkReady 信号是一个关键的广播信号，当主存数据就绪时，它会广播给所有待写入组。每个组的写入使能条件由 BlkReady 与各自的 group 信号进行逻辑 AND 操作决定（例如，对于组 0 是 BlkReady AND group0；对于组 1 是 BlkReady AND group1）。这里的 group0 和 group1 信号是根据地址中的 Index 位选择的结果。这意味着，尽管 BlkReady 是广播的，但只有被 Index 选中的目标组（即其 groupX 信号为高）才会真正使能其写入译码器，从而执行写入操作。

3. 组间协同与状态同步：

所有组共享同一个 LRU 策略实现（通过共享 highestVialdRowNumber 和 highestVialdRowCount 的生成逻辑或类似机制），但每个组独立维护自己的访问历史记录（以确保其内部 LRU 状态机的正确更新）。当某个组执行写入操作时，其状态更新（例如 LRU 位的更新）会通过组间同步信号（图中未明确画出，但这是隐式的）通知相关组，确保替换策略的一致性。数据总线采用多路复用设计，可以同时支持多个组的并行写入（如果控制器允许，例如在不同时钟周期内对不同组进行写入，或者通过更复杂的数据路径实现），大幅提升了整体吞吐量。这种设计既保留了单组处理的简洁性，又通过巧妙的资源共享机制（如 BlkReady 广播）和分布式控制（如 groupX 选择）实现了多组协同工作，使 4 路组相联 Cache 能够高效处理密集的访存请求，优化了写操作的性能和协调性。

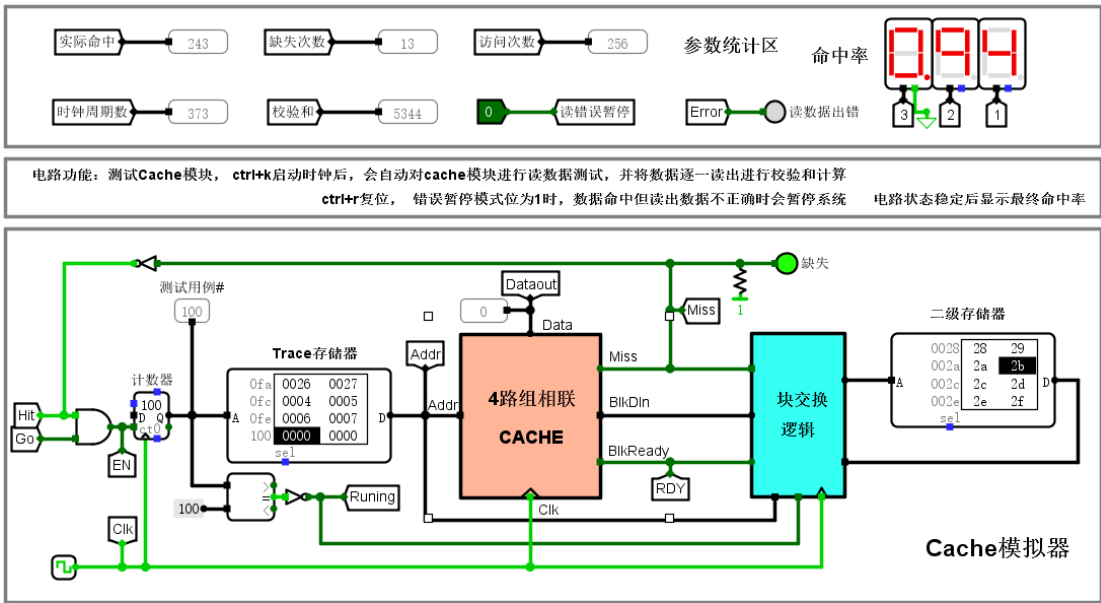


2.6 实验数据测试

EduCoder 平台通过的截图：

6 4路组相联cache设计 2025-06-27 22:05 26306 2 2025-06-28 01:01 1分 20秒 否 700/700 15.00/15 15.00

Cache 自动测试电路的运行截图：



如果设计了不同的映射方式，可以将该电路中cache模块替换为对应Cache模块即可！

默认trace是顺序访问，所以测试系统中判断读错误的逻辑是地址和数据相等，如果更换了trace，这个逻辑不正确

实验三、2 路组相联 cache 设计

3.1 实验目的

本实验旨在掌握 2 路组相联 Cache 的基本原理与设计方法。组相联 Cache 通过折中全相联与直接映射的优点，既降低了硬件复杂度，又保持了较高的命中率。实验需实现以下目标：

1. 地址划分与映射机制：理解主存地址如何划分为标记（Tag）、组索引（Index）和块内偏移（Offset），明确数据块在 Cache 中的组内灵活映射特性。
2. 并行比较与命中判断：设计 2 路并行的组内比较逻辑，通过同时比较组内所有行的标记位，快速判断数据是否命中。
3. 替换算法实现：在组内冲突时，应用 LRU（Least Recently Used）算法选择替换目标，优化 Cache 利用率。

3.2 主要任务

为实现上述目标，除了构建基础 Cache 槽并提供读出写入等信息接口以外，

实验分还需要完成以下核心任务：

1. **地址划分设计**：将 16 位主存地址划分为 12 位标记(Tag)、2 位组索引(Index) 和 2 位块内偏移(Offset)，确保组索引能定位 2 个候选 Cache 行。
2. **组内并行比较**：依照 CAM 设计并行比较逻辑电路，实现组内标记的并行匹配，生成命中信号(Hit)和行选择信号(Line Select)。
3. **LRU 替换策略**：为每组维护一个 Count 计数器，记录行访问顺序，替换时选择组内最久未使用的行。
4. **数据读写整合**：在 Logisim 中构建由 4 组×2 行组成的存储阵列，集成标记存储、数据块存储、比较电路及 LRU 更新逻辑，形成完整 Cache 系统。

3.3 实验方案的总体设计

为了实现 2 路组相联 cache 高速缓冲存储器，我设计了下面的五个模块，分别对应了上述提到的具体任务：

- 构建 Cache 槽，实质是相连存储器。Cache 负责存储数据块和保存辅助信息，如写使能信号、Valid 位、命中信号 Li 等，并提供控制读取和辅助信息的接口。
- 实现相连存储器 CAM 的多路并发比较逻辑。比较地址中的 Tag 和每一个 Cache 槽的有效位信息和 Tag_i 。
- 实现读命中逻辑。当内存地址命中 Hit 时，直接从 Cache 中读出。
- 实现读缺失逻辑。当内存地址没有命中 Miss 时，需要报告读缺失 Miss；同时还需要实现 LRU 算法，找到符合条件的槽，并交给块写入逻辑实现写操作。
- 实现块写入逻辑。收到 Miss 信号时，要在对应的存储块中写入新的数据块。

通过这五个模块的紧密协作，4 路组相联 Cache 能够实现高效的数据存储、查找、命中处理、缺失处理及数据更新，从而实现 4 路组相联 Cache 的主要功能。



2 路组相联和全相联的区别（仅考虑这里的四组 2 路组相连）在于，在判断读缺失逻辑时，只需要考虑某一组内的替换位置进而在块写入时也只需要考虑在某一组内的写入位置。这样的设计使得组相联相比全相联在少量比较器下仍保持

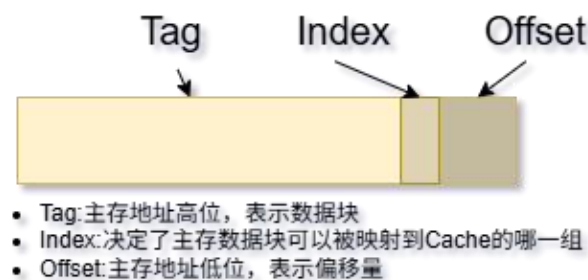
较高命中率，适合现代 CPU 设计。

3.4 实验的实现过程

本实验严格遵循**组相联 Cache**的实际工作流程进行设计与实现，各功能模块的开发顺序与 Cache 处理访存请求的时序逻辑完全匹配。从**存储结构构建**开始，逐步完成**地址解析**、**组选择**、**并行比较**、**命中判断**、**缺失处理**及**替换策略**等核心功能，整个实现过程模拟了真实硬件的工作机制。这样的实现顺序不仅保证了模块间的逻辑一致性，也使我能够直观理解 Cache 从接收访存请求到完成数据访问的完整流程。每个阶段的实现均对应 Cache 实际工作的关键环节，确保最终系统能准确反映硬件行为。实现流程与全相联 Cache 基本一致，可参照 1.4 中的工作流程图，它为理解组相联映射原理提供了清晰的参考。

Cache 的核心功能仍是**通过地址快速定位数据**。

在 4 路组相联设计中，每个 16 位地址被划分为：



2 路组相联 Cache 的实现步骤可以参照 4 路组相联的实现。在这次课设中，全相连的实现时长大约是 4 路组相联的两倍，4 路组相联的实现时长又大约是 2 路组相联的两倍。

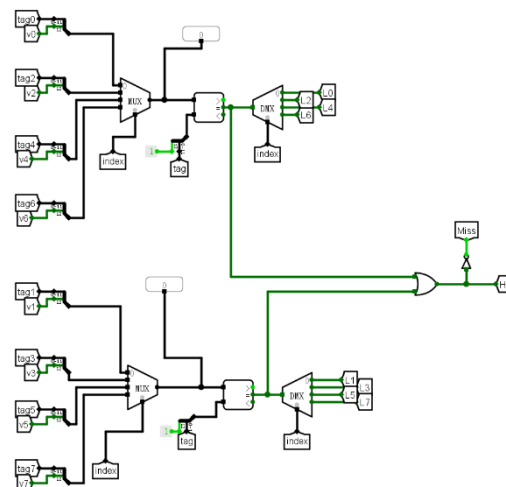
每一步的详细实现会在`3.5 电路图及相关说明`中结合电路图说明。

3.5 电路图及相关说明

2.5.1 Cache 存储部分基本结构

与之前一致。

3.5.2 CAM 及其并发查找逻辑



该电路负责检查请求的数据块是否存在于 Cache 中（命中），如果存在，则检索其对应的 Tag 并验证其有效性。如果数据块未找到，则生成缺失（Miss）信号。

1. Tag 存储与比较：

tag0 到 tag7 信号代表 8 个 Cache 行中存储的 Tag 值。v0 到 v7 信号代表每个 Cache 行的有效位（Valid Bit）。只有当 v 位被置位时，Cache 行才被认为是有效的。电路中使用两个大型的多路选择器 MUX。每个 MUX 接收四个 Tag 输入（例如，顶部 MUX 接收 tag0, tag2, tag4, tag6）。index 信号用于从总共 8 个 Cache 行中选择与当前访问地址对应的 Tag 对（以及它们对应的有效位）。由于是 4 组 2 路的 Cache 结构，index 会选择相关的组。MUX 的输出会与传入的 tag 通过比较器进行比较。MUX 中连接到 0 的输入表示在 index 选择到不存在的路或无效条件时，会选择一个默认值 0。

2. 有效位检查：

有效位（v0 到 v7）也以类似 Tags 的方式被多路复用，确保只有有效的 Cache 行才参与命中判断。

3. 命中信号生成：

Tag 比较器的输出（每个组中每个路的比较结果）与相应的有效位进行逻辑组合。在所选组内，两路的比较结果通过一个或门连接。如果所选组中的任一路的 Tag 匹配且有效位为真，则会生成 Hit 信号。Miss 信号是 Hit 信号的非，通过反相器实现。

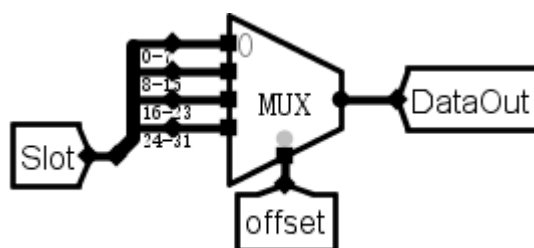
4. 数据输出解复用器 (DMX):

存在两个解复用器（DMX）组件，它们用于控制所选组中两路的数据输出。index 信号驱动这些 DMX，将 Tag 比较和有效位检查的结果导向到相应的 Cache 数据行线（L0-L7）。这些信号（L0 到 L7）代表了每个 Cache 行的“命中”或“被选中”状态，随后将用于使能从该特定行读取数据。

3.5.3 读命中逻辑

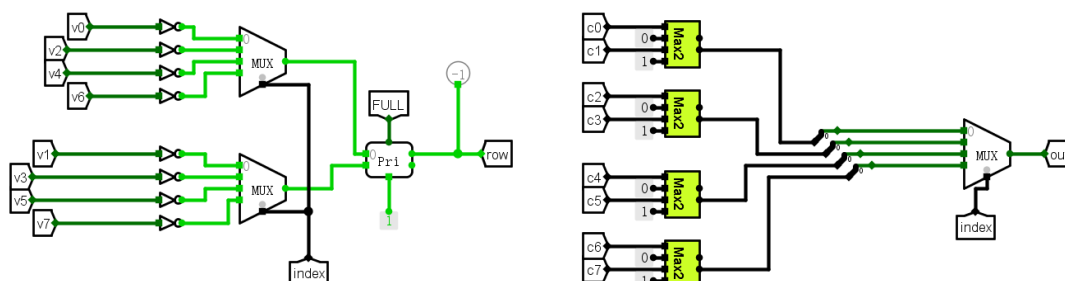
与先前一样，使用分线器分出数据，再根据 Offset 值选择，依靠多路选择器实现。

如下图所示：



2.5.4 读缺失逻辑

下面的电路用于实现读缺失逻辑，也就是未命中时的电路逻辑，需要根据 LRU 算法找到最合适替换的 cache 槽，以信号的形式传递给块写入逻辑，再由块写入逻辑负责写入操作。



上图实现了 Cache 中用于实现 LRU 算法以及查找空闲槽位的核心逻辑。该电路的主要目标是，在需要进行数据写入时，确定在当前 Cache 组中，应该选择哪一个 Cache 行进行替换，或者是否存在并选择一个空闲的槽位。

电路的输入是八个有效位（v0 到 v7），每个 v 信号代表一个 Cache 行的有效状态。值得注意的是，这些 Valid 信号在进入多路选择器 MUX 之前都经过了反相器，这意味着电路实际上是在寻找无效的 Cache 行，也就是空闲的槽位。

接下来是两个多路选择器，它们都由 index 信号控制。index 信号用于选择当前正在操作的特定 Cache 组。由于是二路组相联 Cache，每个组包含两个路。这两个 MUX 分别负责处理对应于所选组的两路的有效位。例如，一个 MUX 负责偶数索引的有效位（如 v0, v2, v4, v6），而另一个 MUX 负责奇数索引的有效位（如 v1, v3, v5, v7），然后 index 信号从中选择出当前组对应的两个有效位状态。

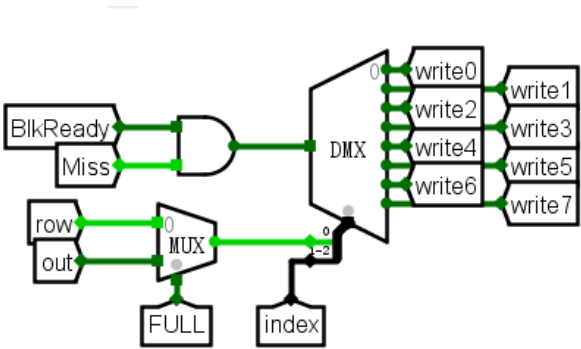
这两个 MUX 的输出随后作为输入，送给一个优先级编码器（Pri）。优先级编码器是一种组合逻辑电路，它根据输入信号的优先级（通常是最低或最高索引的有效输入）来产生一个二进制输出。在这里，Pri 编码器是在寻找第一个为“无效”（即反相后为真）的 Cache 行，以识别可用的空闲槽位。

优先级编码器有两个重要的输出信号：

1. FULL：这个信号指示所选的 Cache 组是否已经完全被有效的 Cache 行占据。如果 Pri 编码器没有找到任何无效的（即空闲的）行，那么 FULL 信号就会被置高，表示该组已满，此时需要执行替换策略。
2. row：这个信号是 Pri 编码器的主要输出，它提供了一个索引值。如果存在空闲槽位，row 就指示了第一个空闲槽位的索引。如果组已满，row 则指示根据替换策略（例如 LRU）应该被替换的 Cache 行的索引。

总之，该电路通过检查 Cache 行的有效位，结合 index 信号来定位特定组，并利用优先级编码器高效地确定当前组的状态（是否已满）以及需要在需要写入新数据时应选择替换哪一行或使用哪个空闲槽位。

3.5.5 块写入逻辑



这个子电路实现了 Cache 的写入使能逻辑，其核心功能是控制哪个具体的 Cache 行（从 write0 到 write7）将被写入数据，这发生在 Cache 缺失并需要从主存加载新数据块时。电路的起始是一个与门，它接收两个关键输入：BlkReady 和 Miss。BlkReady 信号表示从主存读取的数据块已经准备就绪，可以被写入 Cache；而 Miss 信号则表明 CPU 请求的数据在 Cache 中并未找到。只有当这两个条件都满足时（即数据已就绪且发生了缺失），与门的输出才为真，从而使能后续的写入操作。

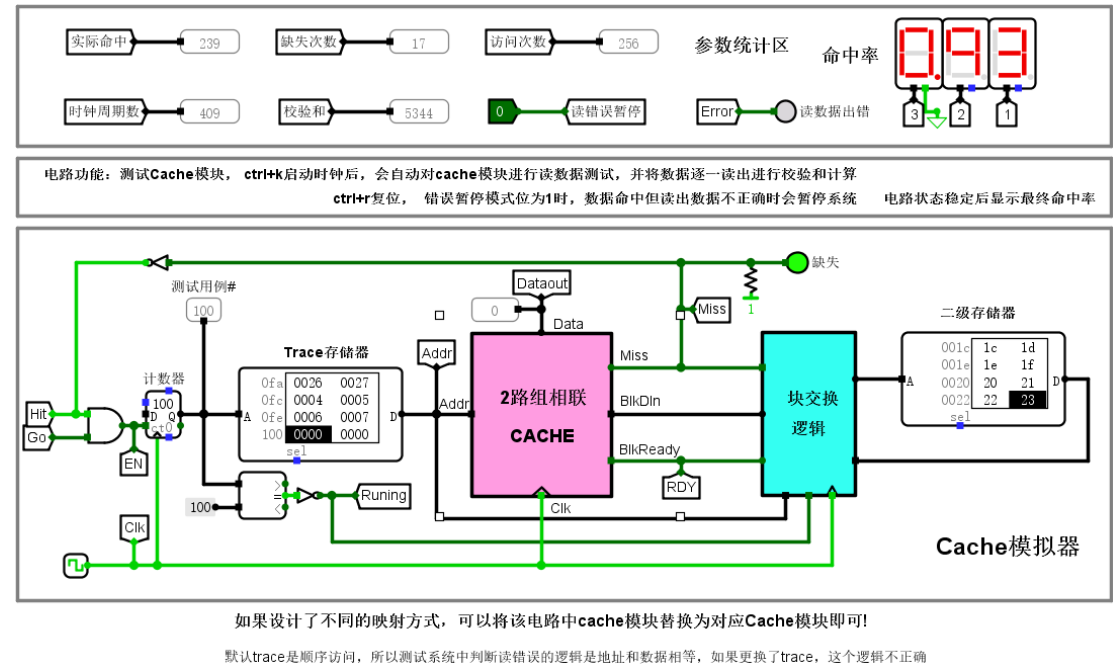
与门之后，是一个多路选择器 MUX，它负责根据当前 Cache 组的状态选择正确的写入目标行。这个 MUX 有两个数据输入：row 和 out，并由 FULL 信号作为选择控制端。FULL 信号指示当前选定的 Cache 组是否已经完全被有效数据块占据（即没有空闲的槽位）。如果 FULL 为高，表示组已满，此时 MUX 将选择 row 信号作为写入目标，这个 row 信号通常来源于 LRU 算法或其他替换策略的逻辑，指示应替换掉哪一个现有的 Cache 行。相反，如果 FULL 为低，表示组中还有空闲槽位，MUX 将选择 out 信号作为写入目标，这个 out 信号通常指示了第一个可用的空闲槽位的索引。

多路选择器选择出的最终目标行索引（从 0 到 7，代表总共 8 个 Cache 行中的一个）随后作为控制信号输入到一个解复用器 DMX。这个 DMX 的输入是来自与门的使能信号。根据 DMX 的控制输入（即选定的目标行索引），它会激活其对应的单个输出线（write0 到 write7 中的一条）。当这条输出线被激活时，就意味着对应的 Cache 行被选中，可以接收从主存加载的新数据块，从而完成写入操作。整个电路的设计确保了在 Cache 缺失发生时，数据能够被准确无误地写入到 Cache 中选定的正确位置。

3.6 实验数据测试

EduCoder 平台通过的截图

Cache 自动测试电路的运行截图



实验四、单周期 MIPS CPU 设计

4.1 实验目的

构建一个 32 位 MIPS CPU 处理器，这里使用单周期硬布线实现，该处理器应支持核心指令集中列出的所有指令。最终设计完成的 CPU 应能运行标准测试程序。下表列出了需要支持的汇编指令。

#	指令	格式	备注
1	Add	add \$rd, \$rs, \$rt	指令功能及指令格式 参考 MIPS32 指令集
2	Add Immediate	addi \$rt, \$rs, immediate	
3	Load Word	lw \$rt, offset(\$rs)	
4	Store Word	sw \$rt, offset(\$rs)	
5	Branch on Equal	beq \$rs, \$rt, label	
6	Branch on Not Equal	bne \$rs, \$rt, label	
7	Set Less Than	slt \$rd, \$rs, \$rt	
8	syscall (display or exit)	syscall	系统调用，用于停机

4.2 主要任务

单周期硬布线 CPU 功能部件：

- (1) 指令计数器 PC，用于存放当前指令执行位置的地址；
- (2) 指令存储器 IM，储存 CPU 要执行的指令；
- (3) 数据存储器 DM，根据设计需求，所有指令都必须在一个时钟周期之内完成，只采用一个存储器不可能在一个时钟周期内同时完成对指令和数据的操作；
- (4) 立即数扩展器 S-EXT，用于将 I 型指令中的 16 位立即数扩展为 32 位；
- (5) 控制器，产生控制信号，控制指令执行的数据通路；
- (6) 寄存器堆 RegiFile，提供 32 个 MIPS 通用寄存器；
- (7) 算术逻辑单元 ALU，产生运算结果。

4.3 实验方案的总体设计

本节将详细阐述本次实验的总体设计，包括实验方案的选择与规划，以及系统的整体架构。我选择基于现有的 MIPS 单周期 CPU 模型进行仿真与分析。该模型清晰地展示了指令的取指、译码、执行、访存和写回五个阶段，以及各阶段所需的硬件组件（如 PC、指令存储器、寄存器堆、ALU、数据存储器等）及其相互连接方式。相较于多周期或流水线 CPU 设计，单周期 CPU 在复杂性上更低，更适合作为初学者理解 CPU 工作原理的起点。

4.3.1 实验规划

实验规划主要围绕以下几个方面展开：

1. 指令集选择与测试用例设计：针对 MIPS 指令集中的核心指令（如 add, sub, lw, sw, beq, j 等）设计测试用例。这些测试用例将覆盖数据通路的所有关键部分，并验证控制逻辑的正确性。例如，对于给定的 `addi $s0, $s0, -1` 指令，我们将重点观察寄存器 \$s0 的值变化、ALU 的运算结果以及写回操作。
2. 观察与分析：在仿真过程中，我们将重点观察以下关键信号和数据：
 - PC 的值变化，以验证指令的顺序执行和分支/跳转逻辑。
 - 指令存储器的输出，确保正确取指。
 - 寄存器堆的读写操作，验证数据读出和写回的正确性。
 - ALU 的输入、操作码和输出，验证算术逻辑单元的功能。
 - 数据存储器的读写操作，验证访存指令的正确性。
 - 控制单元的输出信号，确保其不同指令下能生成正确的控制信号。
3. 结果记录与报告撰写：详细记录仿真结果，并基于观察结果撰写实验报告，分析 CPU 的工作原理和可能存在的问题。

4.3.2 总体设计

本实验所使用的 MIPS 单周期 CPU 的总体设计遵循经典的冯·诺依曼架构，其核心组成部分包括：

1. 程序计数器 (PC)：负责存储下一条待取指令的地址。在每个时钟周期，PC 的值会更新为当前指令地址加 4（对于顺序执行），或者根据分支/跳转指令的结果更新为新的目标地址。

2. 指令存储器 (Instruction Memory): 根据 PC 提供的地址, 取出对应的机器指令。
3. 寄存器堆 (RegFile): 包含 32 个通用寄存器, 用于存储数据和中间计算结果。它支持两个独立的读端口 (R1#, R2#) 和一个写端口 (W#), 以满足指令对多个寄存器的同时访问需求。
4. 算术逻辑单元 (ALU): 执行指令中的算术和逻辑运算 (如加、减、与、或等)。其操作类型由控制单元的 ALUOp 信号决定。
5. 数据存储器 (Data Memory): 用于存储和读取数据。访存指令 (lw, sw) 通过它来与外部存储器交互。
6. 控制单元 (Control Unit): 是 CPU 的“大脑”, 根据当前指令的 OpCode 和 Func 字段, 生成控制整个 CPU 各个组件 (如 ALUOp、MemToReg、MemWrite、RegWrite、Branch、Jump 等) 的信号。
7. 多路选择器 (MUX): 用于根据控制信号选择不同数据源, 例如选择 ALU 的第二个输入、写回寄存器的数据来源、以及 PC 的下一个值。
8. 符号扩展单元 (Sign Extend): 将 16 位立即数扩展为 32 位, 用于 addi、lw、sw 等指令。
9. 左移 2 位单元 (Shift Left 2): 用于 PC 相对寻址, 将分支偏移量左移两位, 使其成为字地址偏移。

这些组件通过数据通路和控制信号线相互连接, 形成一个完整的指令执行流程: 指令从指令存储器取出, 经过译码后, 相关数据从寄存器堆读出, 送入 ALU 进行运算, 结果可能写入数据存储器或写回到寄存器堆。PC 的更新确保了指令的连续执行或流程控制的跳转。整个设计确保了每条指令在一个时钟周期内完成所有操作。

4.4 实验的实现过程 (结合电路图说明)

4.4.1 指令解析

将 32 位输入操作码用分线器接出, 最高 6 位为操作码 op, 21-25 位为 rs 寄存器编号, 16-20 位为 rt 寄存器编号、11-15 位为 rd 寄存器编号, 0-5 位为功能码 func; 取低 16 位, 作为 I 型指令中的立即数操作数。单周期和多周期指令译码方式相同, 如下图所示:

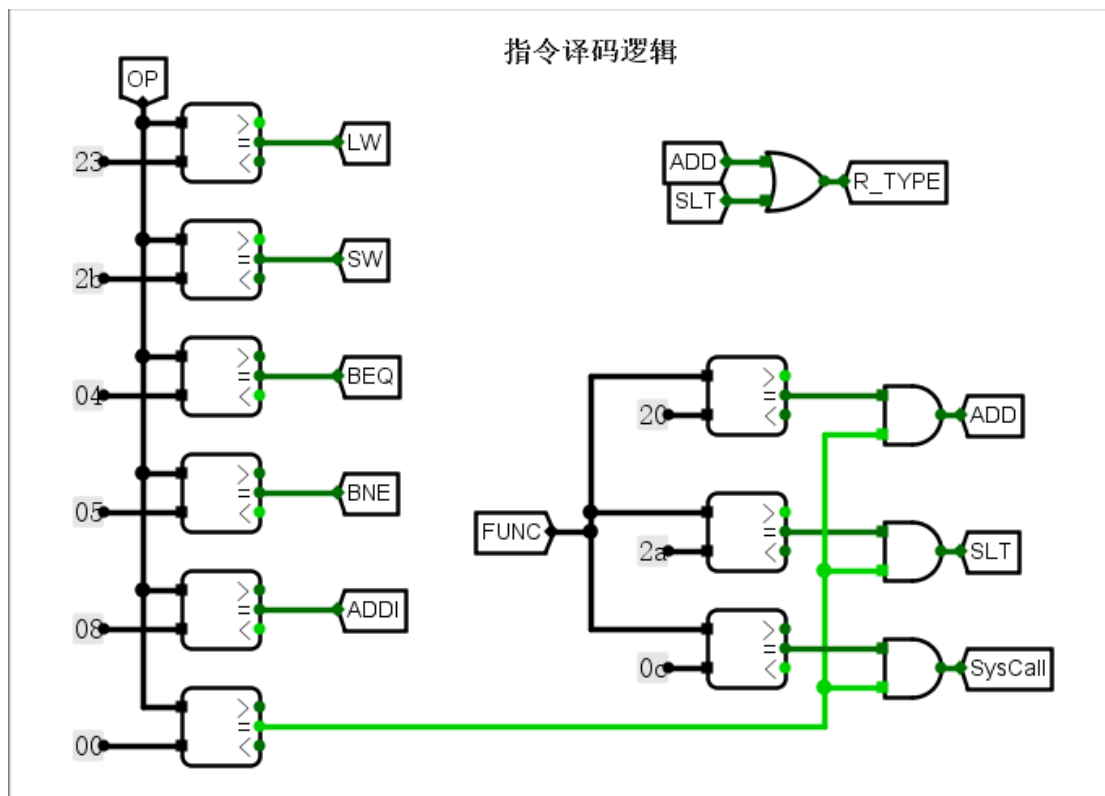


图 4.4.1.1 指令译码

4.4.2 实现地址转移逻辑

PC+4 接到 MUX 的 0 输入端, 16 位立即数扩展结果左移 2 位, 加上 PC+4, 接到 MUX 的 1 输入端, 选择信号为 branch。实验要求的分支转移指令为 beq 和 bne, 两条指令均为 I 型指令, 由于两条指令的特殊性, 可利用 ALU 的 equal 信号, 最终 $\text{branch} = \text{beq} \times \text{equal} + \text{bne} \times (\neg \text{equal})$ 形成分支信号

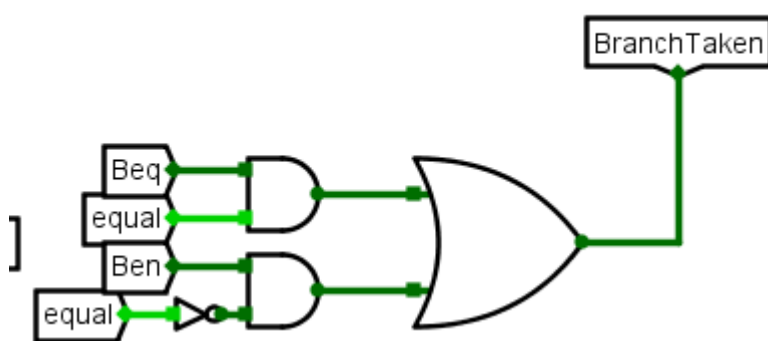


图 4.4.1.2 地址转移逻辑

4.4.3 单周期硬布线控制器

(1) 数据通路综合

构建数据通路。绘制主要功能部件输入来源表, 该表主要用于描述控制类信

号，仅保留数据类信号，具体如表 4.4.3.1，最左侧为指令助记符，第一行为数据项。输入源合并。

表 4.4.3.1 单周期功能部件输入来源表

指令	PC	IM	RegiFile (RF)				S-EXT	ALU		DM	
			R1#	R2#	W#	Din		A	B	Addr	Din
add	PC+4	PC	rs	rt	rd	ALU		RF.D1	RF.D2		
slt	PC+4	PC	rs	rt	rd	ALU		RF.D1	RF.D2		
addi	PC+4	PC	rs		rt	ALU	IMM[15:0]	RF.D1	S-EXT		
lw	PC+4	PC	rs		rt	DM.Dout	IMM[15:0]	RF.D1	S-EXT	ALU	
sw	PC+4	PC	rs	rt			IMM[15:0]	RF.D1	S-EXT	RF.D2	ALU
beq	PC+4+offset/PC	PC	rs	rt			IMM[15:0]	RF.D1	RF.D2		
bne	PC+4+offset/PC	PC	rs	rt			IMM[15:0]	RF.D1	RF.D2		
syscall	PC	PC									
合并	2输入	1输入	1输入	1输入	2输入	2输入	1输入	1输入	2输入	2输入	1输入

- (2) 列出所有功能部件、多路选择器控制信号、运算操作选择的产生条件，如表 1.3 所示，横坐标给出的是不同指令的译码信号，表中有 1 的位置表示当前指令会产生对应的信号，利用译码电路生成各指令译码信号，然后以行为单位将各个产生信号的条件相加（逻辑或）即可得到控制信号的逻辑表达式。

■RegDst 为 1 表示 RegiFile 写回地址由 R 型指令 rd 字段给出，否则由 I 型指令 rt 字段给出；

■RegWrite 为 1 打开 RegiFile 写使能，表示数据写回 RegiFile；

■AluSrcB 为 1 表示 ALU 的第二个操作数将由立即数扩展器 S-EXT 给出，否则由 RegiFile 的第二个输出给出；

■bne 为 1 表示指令为 bne；

■beq 为 1 表示指令为 beq，beq 与 bne 信号与 ALU equal 信号结合可控制 PC 数据来源；

■MemWrite 为 1 打开数据存储器 DM 写使能，将 ALU 运算结果写入数据存储器 DM；

■MemToReg 为 1 表示从数据存储器 DM 中选数据送入 RegiFile；

■Halt 为停机信号，为 1 时系统停机；

■加法信号，表示 ALU 应对两个操作数执行加法操作；

■比较信号，表示 ALU 应对两个操作数执行比较操作。

- (3) 根据以上分析，可最终形成如下表所示指令与信号对应关系表，进而绘制出单周期硬布线控制器电路如下：

指令	RegDst	RegWrite	MemToReg	MemWrite	AluSrc	Beq	Bne	Halt
R_TYPE	1	1	0	0	0	0	0	0
ADDI	0	1	0	0	1	0	0	0
LW	0	1	1	0	1	0	0	0
SW	0	0	0	1	1	0	0	0
BEQ	0	0	0	0	0	1	0	0
BNE	0	0	0	0	0	0	1	0
SysCall	0	0	0	0	0	0	0	1

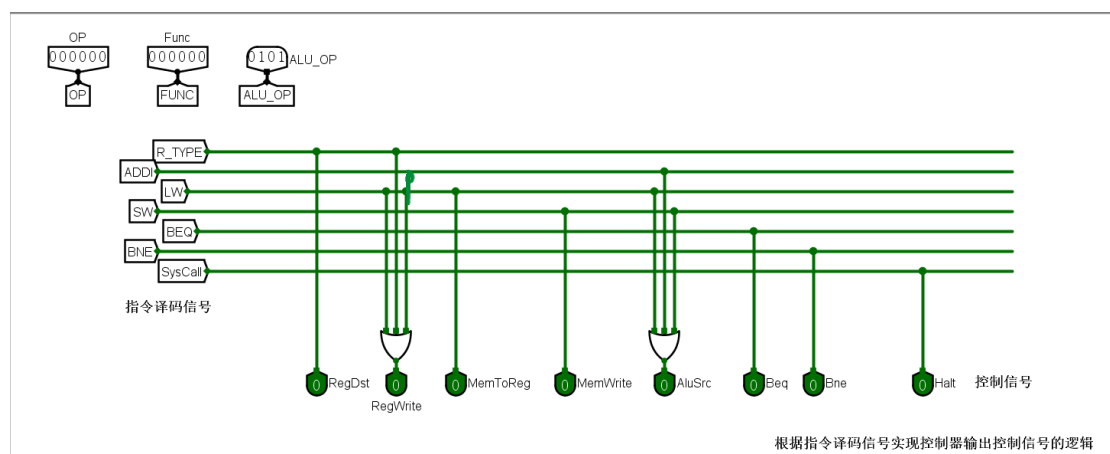


图 4.4.3.1 MIPS 单周期 CPU 控制器电路

4.4.4 单周期 CPU 总体结构图

通过上面的分析，我们已经得到单周期 CPU 的所有功能部件，我们也知道了所有指令的数据通路，以及如何产生控制信号控制指令数据的传递，因此把这些部件和控制信号连接起来就得到了单周期 CPU 总体结构图。

【主要参考图 4.4.4.2】

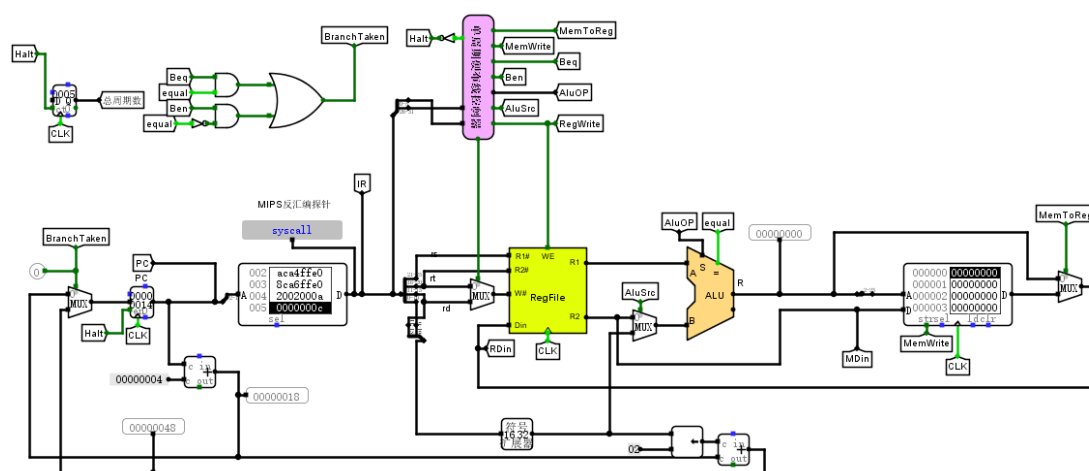
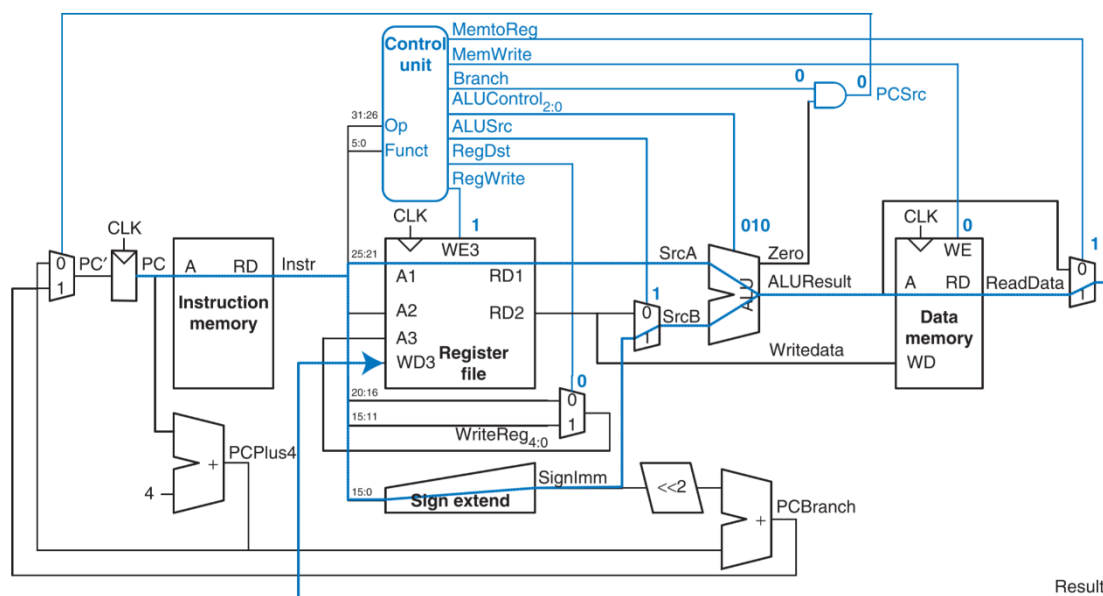
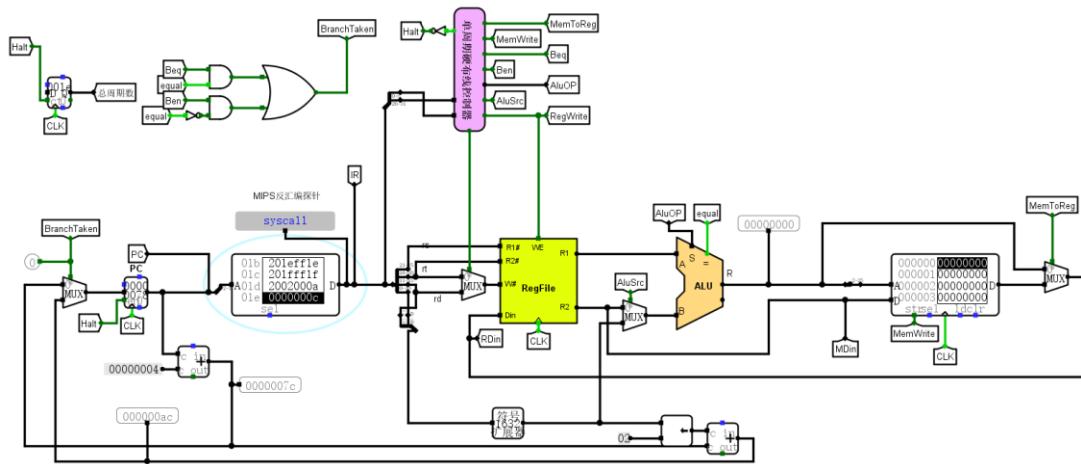
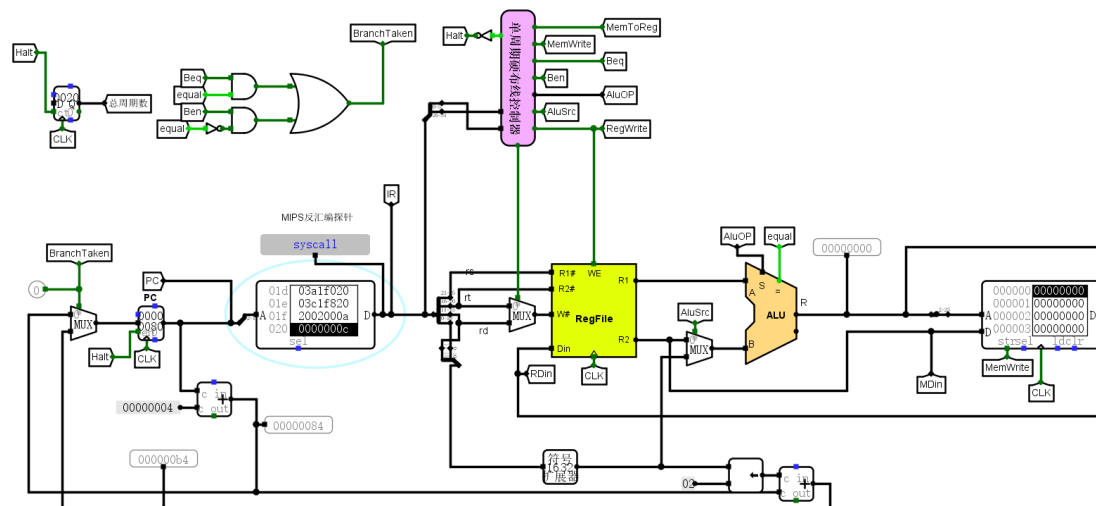


图 4.4.4.1

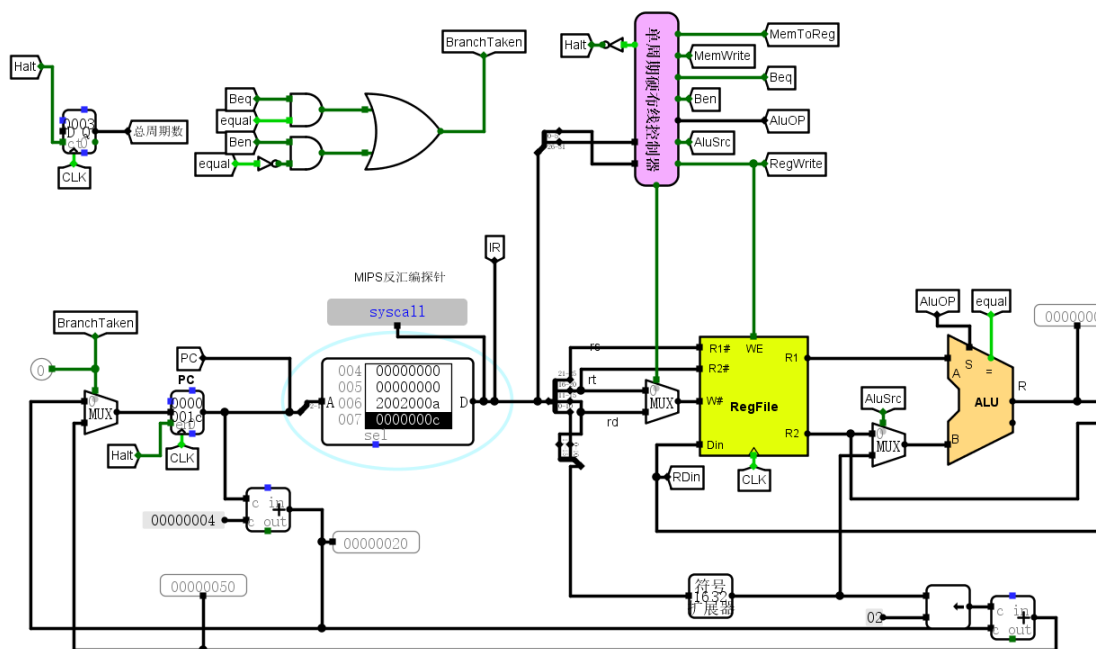




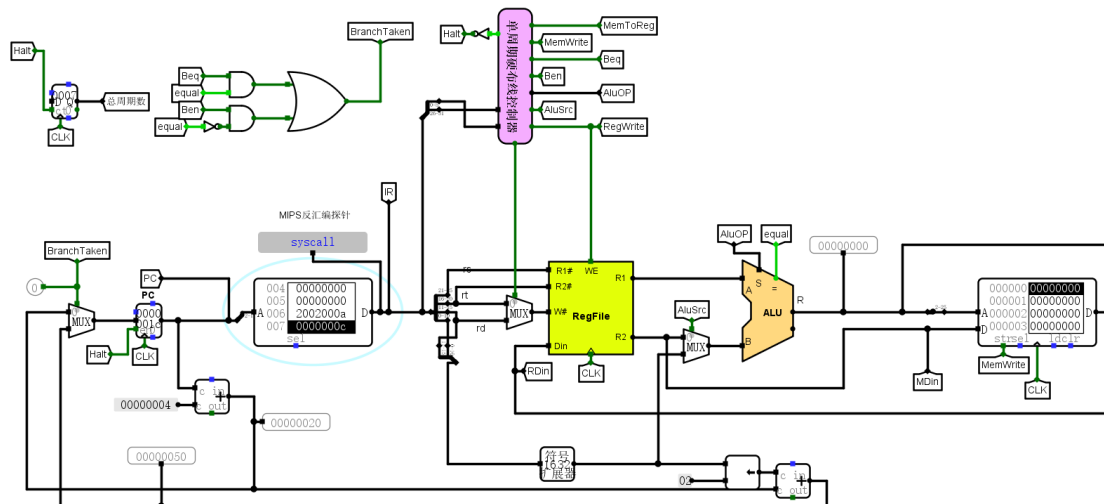
3. Add



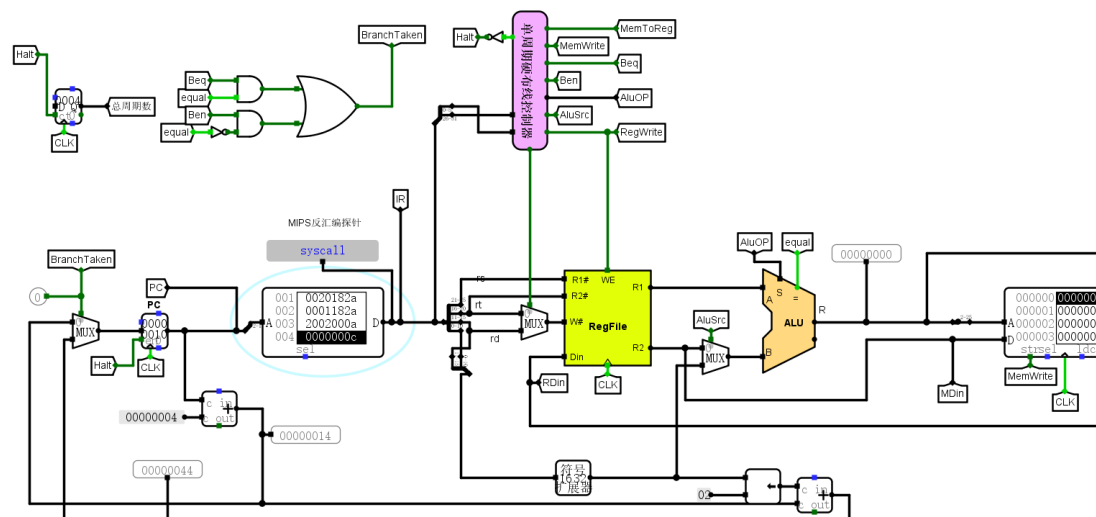
4. Beq



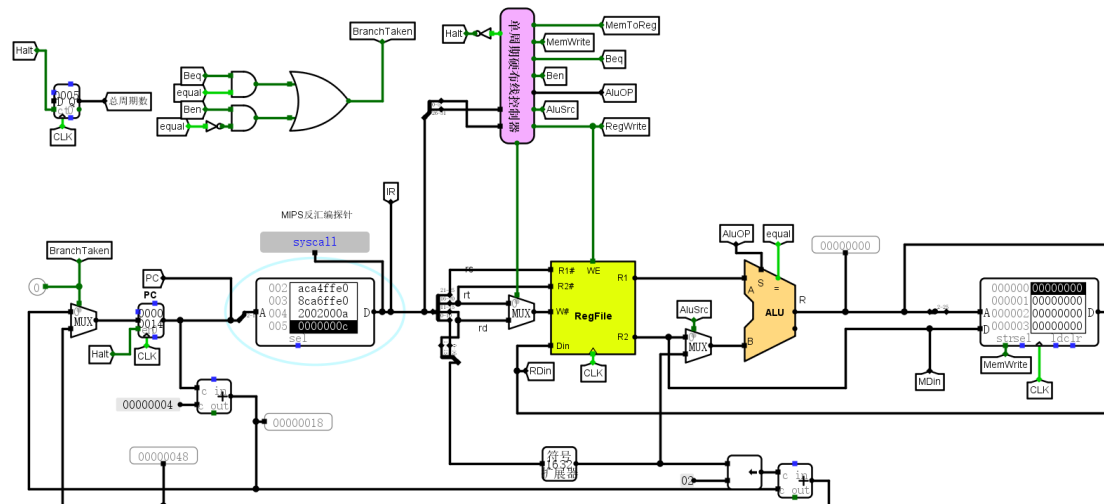
5. Bne



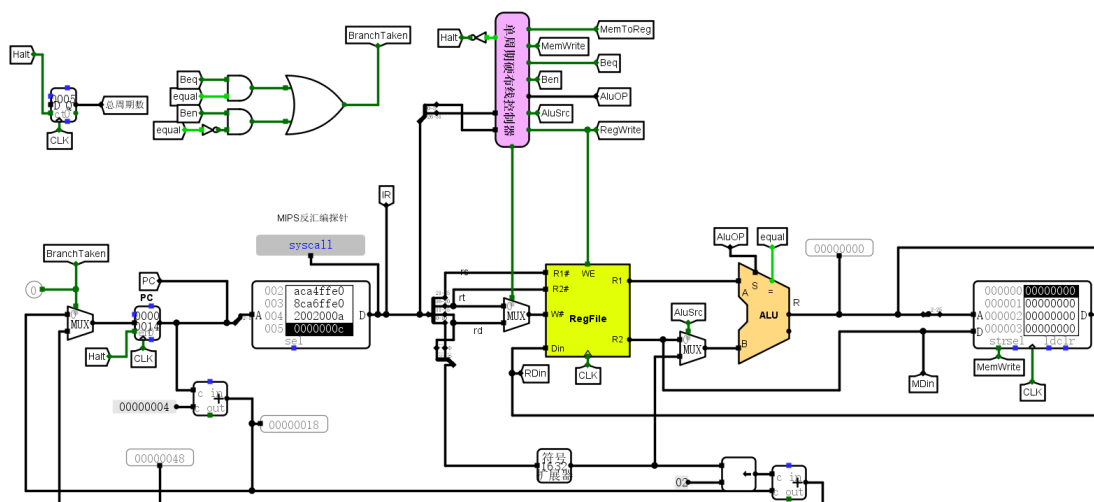
6. Slr



7. Sw



8. Lw



实验五、微程序地址转移逻辑设计

5.1 实验目的

本实验是多周期 MIPS CPU（微程序）的准备部分。多周期的 MIPS CPU（微程序）的运行过程中，一条指令会对应执行多条微指令，最后再通过程序计数器 PC 跳转到下一条指令。对应【图 6.49】中红色矩形框标出的部分。

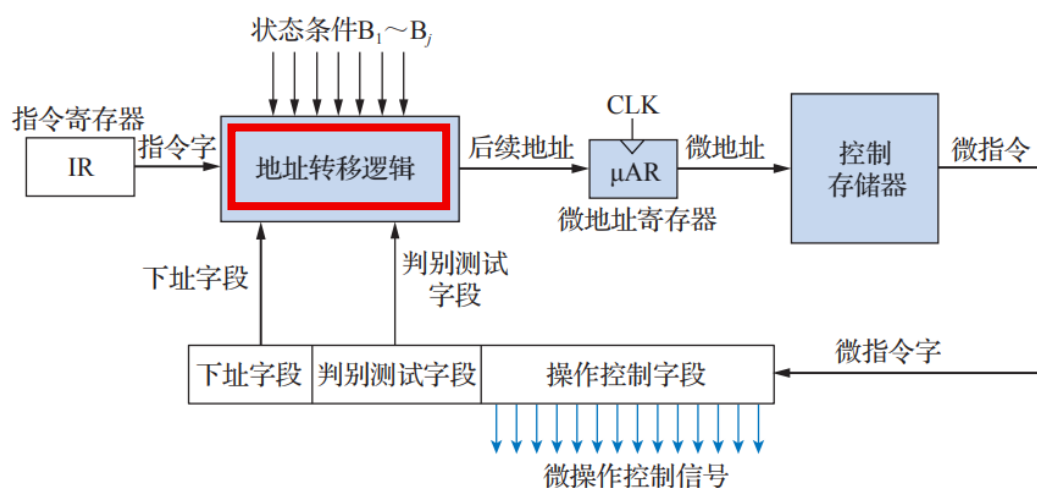


图 6.49 微程序控制器组成框图

5.2 主要任务

【部分见`5.1 实验目的`】

1. 明确需要建议的双射关系两端
2. 在下表中找到指令与微指令的转换关系

3. 导入 Excel 表格中，生成相关函数表达式和对应机器码
4. 将相关函数表达式和对应机器码保存到电路存储控制器中。

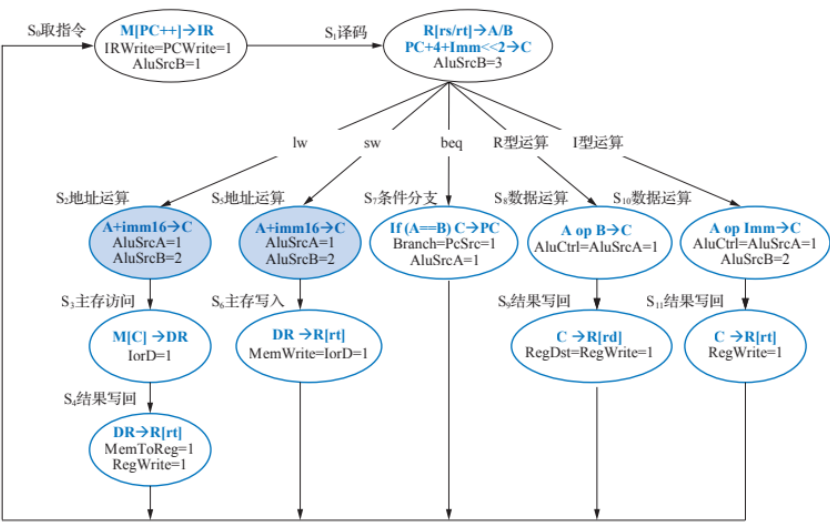


图 6.46 多周期 MIPS 指令执行状态转换图

5.3 实验方案的总体设计

逐步实现`5.2 主要任务`中的主要任务。

5.4 实验的实现过程（结合电路图说明）

5.4.1 微程序地址转移逻辑自动生成

在【图 6.46】中找到下表左侧机器指令译码信号对应的 10 进制入口地址，Excel 会自动在【工作表 2】中为每一条指令生成对应逻辑表达式【见下方蓝色区域】。

机器指令译码信号							微程序入口地址				
R_Type	ADDI	LW	SW	BEQ	BNE	SYSCALL	入口地址 10进制	S3	S2	S1	S0
1							7	0	1	1	1
	1						11	1	0	1	1
		1					2	0	0	1	0
			1				5	0	1	0	1
				1			9	1	0	0	1
					1		10	1	0	1	0
						1	13	1	1	0	1

实验六、MIPS 微程序 CPU 设计

6.1 实验目的

1. 掌握多周期 MIPS CPU 设计原理
2. 掌握微程序控制器设计的基本原理
3. 利用微程序控制器的设计实现多周期 MIPS 处理器

6.2 主要任务

6.2.1 完成指令集设计

在实现多周期 MIPS CPU（微程序控制）时，需要得到每一条指令的机器码和其微指令的对应关系，从而执行对应的微程序。

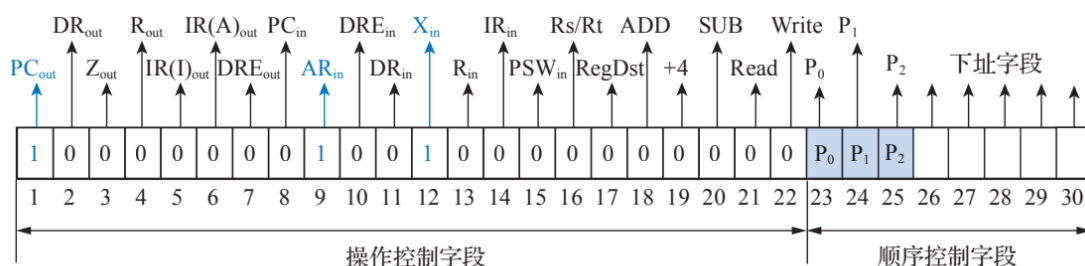


图 6.47 微指令格式示例

如图 6.47 所示, 需要建立操作控制字段和微指令的双射关系。本实验需要利用提供的 Excel 表格 (内嵌双射建立逻辑), 找到该双射关系, 并导入电路中。

6.2.2 完成电路逻辑

上述所有工作的最终目的是：明确每条指令所需执行的微指令及其执行顺序。基于此，接下来需要设计并实现能够按正确顺序执行这些微指令的逻辑电路。对应电路是：1. 多周期 MIPS 微程序；2. 多周期微程序控制器。

最后还需要测试联调。

6.3 实验方案的总体设计

步骤 1: 设计微程序控制器, 包括主要任务 6.2.1, 见【6.2.1 完成指令集设计】和指令译码逻辑等。

步骤 2: 构建 MIPS 主机通路, 即在电路【多周期 MIPS (微程序)】中完成下图所示逻辑通路的连线。

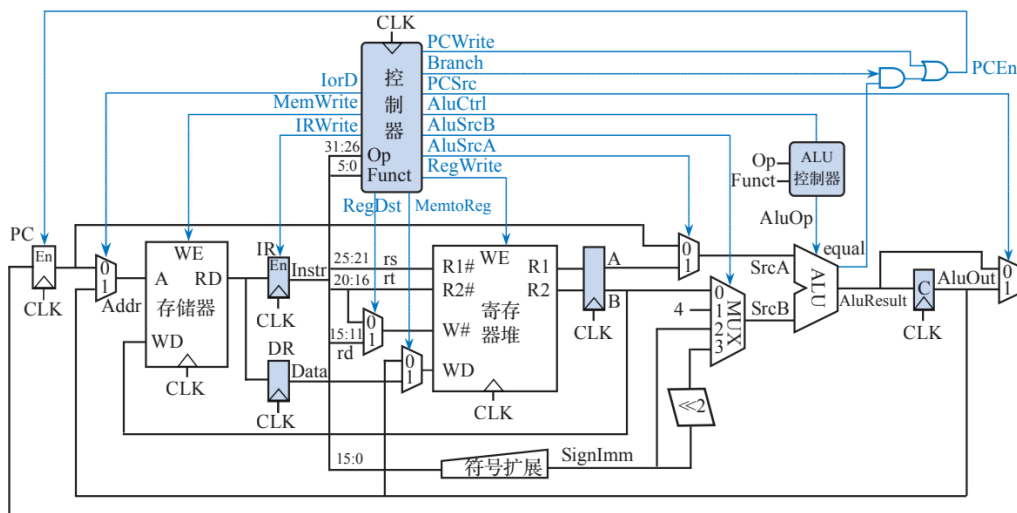


表 6.10 多周期 MIPS 处理器的控制信号及功能

控制信号	信号分类	功能说明
Branch	指令译码信号	分支指令译码信号，beq、bne 指令需要产生类似的译码信号
RegDst	多路选择控制	写入目的寄存器选择，为 1 时写入 rd 寄存器，为 0 时写入 rt 寄存器
AluSrcA	多路选择控制	控制 ALU 的第一输入，为 0 时输入 PC 值，为 1 时输入寄存器 A 的值
AluSrcB	多路选择控制	控制 ALU 的第二输入，值为 0~3 时分别输入 B 值、常量 4、立即数、偏移地址
AluCtrl	多路选择控制	ALU 控制器选择控制信号，为 0 时控制 ALU 做加法；为 1 时 ALU 的操作由指令功能译码决定，如 R 型指令由 funct 字段决定
MemToReg	多路选择控制	为 0 时写入暂存在 C 寄存器中的 ALU 运算结果，为 1 时写入 DR 寄存器的值
PCSrc	多路选择控制	控制程序计数器 PC 的输入，为 0 时输入 PC+4，为 1 时输入分支目标地址
IorD	多路选择控制	控制存储器地址输入，为 0 时输入指令地址，为 1 时输入寄存器 C 中的数据地址
RegWrite	功能部件控制	控制寄存器堆写操作，为 1 时数据需要写入指定寄存器，受时钟驱动
MemWrite	功能部件控制	控制数据存储器写操作，为 0 时进行读操作，为 1 时进行写操作，写受时钟驱动
PCWrite	功能部件控制	控制数据存储器写操作，为 0 时进行读操作，为 1 时进行写操作，写受时钟驱动
IRWrite	功能部件控制	控制指令寄存器写操作，为 1 时有效，受时钟驱动

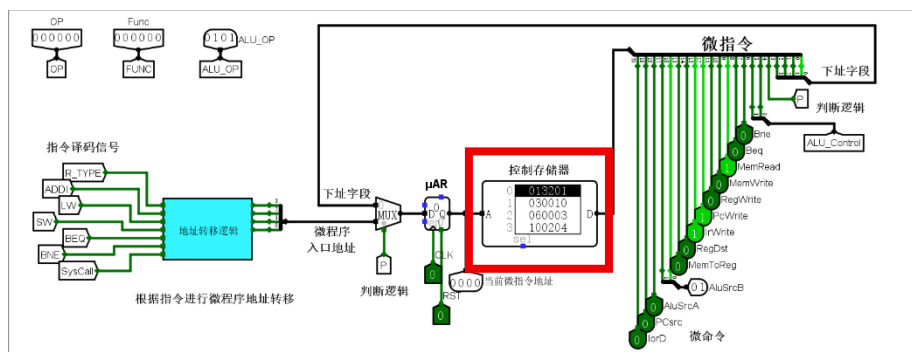
从课本 P211 开始介绍了如下内容：

1. 取指令阶段数据通路
2. 译码 / 取数阶段数据通路
3. R 型算术逻辑运算指令执行阶段的数据通路
4. I 型算术逻辑运算指令执行阶段的数据通路
5. lw 指令执行阶段的数据通路
6. sw 指令执行阶段的数据通路

此处尚未开始连线，只需找出每一条指令需要传递的信号就可以填充完整 Excel 表格【微指令自动生成(2019-4-22).xlsx】，便可以得到如下结果：

微指令功能	状态	微指令地址	IorD	PCSrc	AluSrcA	AluSrcB	MemToReg	RegDst	IrWrite	PCWrite	RegWrite	MemWrite	MemRead	BEQ	BNE	AluControl	P	下址字段	微指令	十六进制
取指令	0	0000	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0001	00010011001000000000	13201
译码	1	0001	0	0	0	11	0	0	0	0	0	0	0	0	0	0	1	0000	00110000000000001000	30010
LW1	2	0010	0	0	0	1	10	0	0	0	0	0	0	0	0	0	0	0011	00110000000000000001	60003
LW2	3	0011	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0100	00000000001000000100	100204
LW3	4	0100	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0000	00001000100000000000	8800
SW1	5	0101	0	0	0	1	10	0	0	0	0	0	0	0	0	0	0	0110	00110000000000000011	60006
SW2	6	0110	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0000	00000000010000000000	100400
R1	7	0111	0	0	0	1	00	0	0	0	0	0	0	0	0	10	0	1000	00100000000000100100	40048
R2	8	1000	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0000	00000010010000000000	4800
BEQ	9	1001	0	1	1	00	0	0	0	0	0	0	0	1	0	0	0	0000	01100000000010000000	C0100
BEN	10	1010	0	1	1	00	0	0	0	0	0	0	0	0	1	0	0	0000	01100000000010000000	C0080
ADDI1	11	1011	0	0	0	1	10	0	0	0	0	0	0	0	0	0	0	1100	00110000000000000110	6000C
ADDI2	12	1100	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0000	00000000100000000000	800
SYSCALL	13	1101	1	0	0	0	0	0	0	0	0	0	0	0	0	11	0	1101	00000000000001101101	10006D

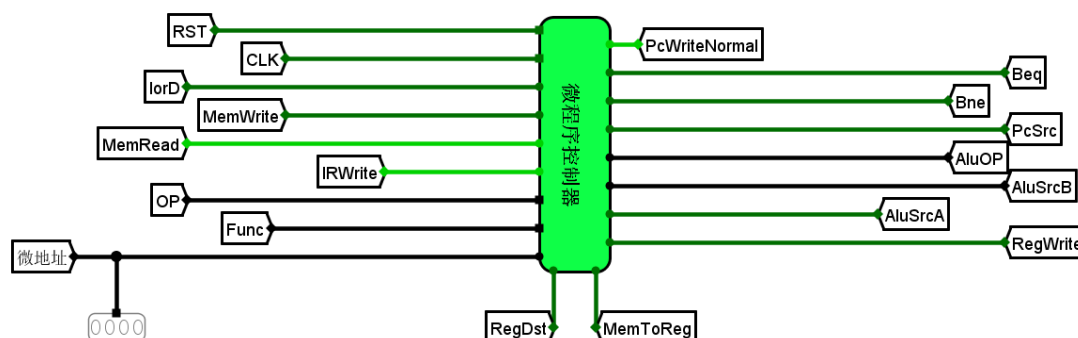
取紫色 16 进制值保存到电路【多周期微程序控制器】中的控制存储器，该 CPU 便具有了编译功能，即可以识别 2 进制机器码（或 16 进制）从而确定将要执行的指令，电路如下图中红色方框所示。



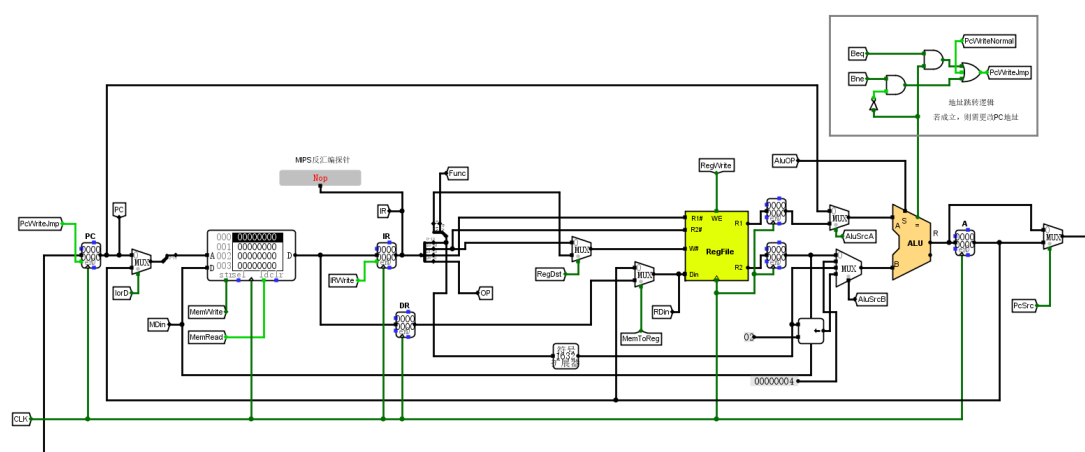
接下来，我深入理解各个指令的含义和要执行的微指令，构建数据通路。

6.4.3 多周期 CPU 总体结构图

经过以上步骤，我可以构建如下微程序控制器接口：



它相当于一个编译器，输入一条指令，经过分线操作，它可以产生需要执行的微指令的信号。接下来，只需要根据【图 6.27 多周期 MIPS 处理器的数据通路高层视图，见`6.3 实验方案的总体设计`】搭建多周期 MIPS 处理器即可。使用了上述接口的设计，可以实现更简洁的连线如下：

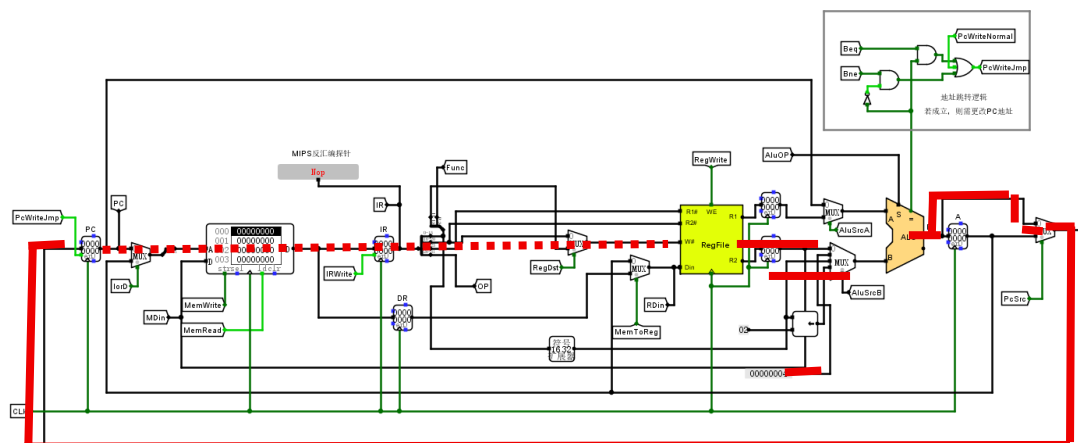


该电路图的具体讲解见`6.5 电路图及其相关说明`

6.5 电路图及其相关说明

6.5.1 取指令阶段数据通路

多周期 MIPS 指令周期的第一个时钟周期 T1 是取指令阶段（Instruction Fetch, IF），该阶段包含两条并行工作的数据通路（如图中红色路径所示）。



在传统的单周期 MIPS CPU 设计中，程序计数器（PC）的自增通常需要一个专用的加法器来完成，以确保在每个时钟周期结束时 PC 能正确指向下一条指令的地址。然而，在多周期 MIPS 的实现中，为了优化硬件资源的使用，PC 的自增功能并未采用独立的加法器，而是复用 ALU（算术逻辑单元）来完成这一计算。这种设计选择虽然增加了控制逻辑的复杂度，但有效减少了硬件开销，使得整体结构更加紧凑。

这一设计与单周期 MIPS CPU 的实现方式有明显区别。在单周期架构中，由于每条指令的执行仅占用一个时钟周期，硬件资源通常较为独立，以确保各功能单元能并行工作。而在多周期架构中，由于指令的执行被拆分为多个阶段，硬件资源（如 ALU）可以在不同阶段被重复利用，从而提高资源利用率。

如果仔细观察最终的电路实现图，可能会对这一设计产生疑问，因为 PC 的自增并未像传统设计那样直接通过加法器完成，而是依赖于 ALU 的运算结果。这种复用策略虽然在逻辑上是可行的，但在电路布局上可能会显得不够直观，尤其是对于习惯了单周期 MIPS 设计的人来说。因此，理解多周期 MIPS 的数据通路和控制逻辑需要更细致的分析。

6.5.2 译码/取数阶段数据通路

第二个时钟周期 T2 是译码/取数阶段，其数据通路如图 6.29 中粗线所示。

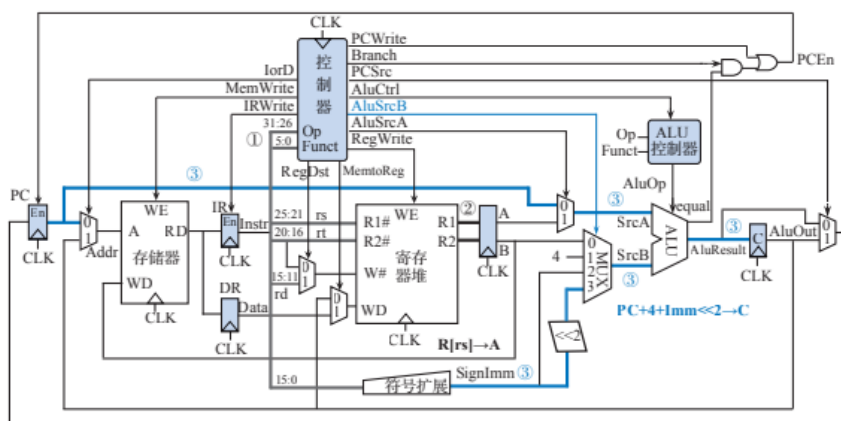


图 6.29 多周期 MIPS 译码取数阶段数据通路

在译码和取数阶段（T2 时钟周期），数据通路的主要操作包括指令译码和取操作数。首先，操作控制器根据指令寄存器（IR）中的指令字进行译码，生成相应的译码信号。接着，从寄存器堆中读取操作数，具体过程是将 IR 中的 rs 和 rt 字段分别连接到寄存器堆的 R1#和 R2#端口，从而选择对应的寄存器值输出到 R1 和 R2 端口。这些值随后被送入 A 和 B 寄存器中暂存，为下一阶段的指令执行提供操作数。

此外，该阶段还会提前计算分支目标地址。通过符号扩展将立即数扩展为 32 位后，与程序计数器（PC）的值相加，结果暂存在 C 寄存器中。如果当前指令是条件分支指令，下一时钟周期可以直接从 C 寄存器获取分支目标地址，避免了等待时间；如果是其他指令，这个地址则不会被使用。

寄存器堆的读操作是通过组合逻辑实现的，不需要额外的控制信号。A 和 B 寄存器的写入操作仅受时钟控制，因此该通路也不需要设置控制信号。在计算分支目标地址时，数据通路复用了 ALU 部件，通过设置 AluSrcA=0、AluSrcB=3 和 AluCtrl=0 来完成加法操作，从而节省了一个加法器的硬件开销。

完成取指令和译码/取数阶段后，指令字被锁存在 IR 寄存器中，rs 和 rt 寄存器的值分别锁存在 A 和 B 寄存器中，分支目标地址则暂存在 C 寄存器中。根据指令译码信号，后续的执行阶段会对这些暂存的数据进行进一步处理，以实现不同类型的指令功能，例如 R 型算术逻辑运算、立即数运算、访存指令和条件分支指令等。

6.5.3 其他指令及其指令执行

其余重要电路逻辑还有如下四点：

1. R 型算术逻辑运算指令执行阶段的数据通路
2. I 型算术逻辑运算指令执行阶段的数据通路
3. lw 指令执行阶段的数据通路
4. sw 指令执行阶段的数据通路

这些都与单周期 MIPS CPU 类似。整体的转移逻辑如下表所示。

表 6.11 MIPS 指令多周期数据通路及控制信号

指令类型	时钟周期	操作	控制信号（非零值）
取指令	T1	M[PC] → IR PC+4 → PC	IRWrite=PCWrite=1 AluSrcB=1
	T2	R[rs] → A; R[rt] → B; PC+Imm<<2 → C	AluSrcB=3
R 型运算	T3	A op B → C	AluCtrl=AluSrcA=1
	T4	C → R[rd]	RegWrite=RegDst=1
I 型运算	T3	A op imm → C	AluCtrl=AluSrcA=1 AluSrcB=2
	T4	C → R[rt]	RegWrite=1
lw 指令	T3	A + imm → C	AluSrcA=1 AluSrcB=2
	T4	M[C] → DR	IorD=1
	T5	DR → R[rt]	MemToReg=RegWrite=1
sw 指令	T3	A + imm → C	AluSrcA=1 AluSrcB=2
	T4	B → M[C]	IorD=MemWrite=1
beq 指令	T3	if (A = B) C → PC	AluSrcA=PCSrc=Branch=1

R 型算术逻辑运算指令执行阶段的数据通路：

在 T1 时钟周期，PC 内容（指令地址）被送入存储器地址寄存器 M[PC]，指令被从存储器中读取到指令寄存器 IR，同时 PC 值更新为 PC+4。接着在 T2 时钟周期，根据 IR 中的 RS 字段，数据从寄存器堆读取到寄存器 A，同时 IR 中的 RT 字段对应的寄存器数据或 PC+Imm<<2 被送入寄存器 B。随后在 T3 时钟周期，ALU 对 A 和 B 中的数据执行运算（A op B），并将结果送入寄存器 C。最后在 T4 时钟周期，C 中的结果被写入寄存器堆中由 IR[rd]指定的目标寄存器。

I 型算术逻辑运算指令执行阶段的数据通路：

T1 和 T2 时钟周期的操作与 R 型运算指令相同，完成取指令和读取寄存器操作。不同的是，在 T3 时钟周期，ALU 对 A 和立即数(Imm)执行运算(A op Imm)，结果送入寄存器 C。接着在 T4 时钟周期，C 中的结果被写入寄存器堆中由 IR[rt]指定的目标寄存器。

lw 指令（load word）执行阶段的数据通路：

其 T1 和 T2 时钟周期的操作也与 R 型运算指令相同，用于取指令和读取寄存器。随后在 T3 时钟周期，ALU 计算出内存地址（A + Imm），并将结果送入寄存器 C。在 T4 时钟周期，C 中的地址 M[C]所指向的数据被送入数据寄存器 DR，完成从内存中读取数据的操作。最终在 T5 时钟周期，DR 中的数据被写入寄存器堆中由 IR[rt]指定的目标寄存器。

sw 指令（store word）执行阶段的数据通路：

T1 和 T2 时钟周期的操作同样与 R 型运算指令相同，用于取指令和读取寄存器。接着在 T3 时钟周期，ALU 计算出内存地址（A + Imm），并将结果送入寄存器 C。最后在 T4 时钟周期，B 中的数据被写入 C 中的地址 M[C]指向的内存位置，完成向内存中写入数据的操作。

这些数据通路均与单周期 MIPS CPU 的工作方式类似，并且表格中也详细列出了在每个时钟周期中需要激活的控制信号。

6.6 实验数据测试

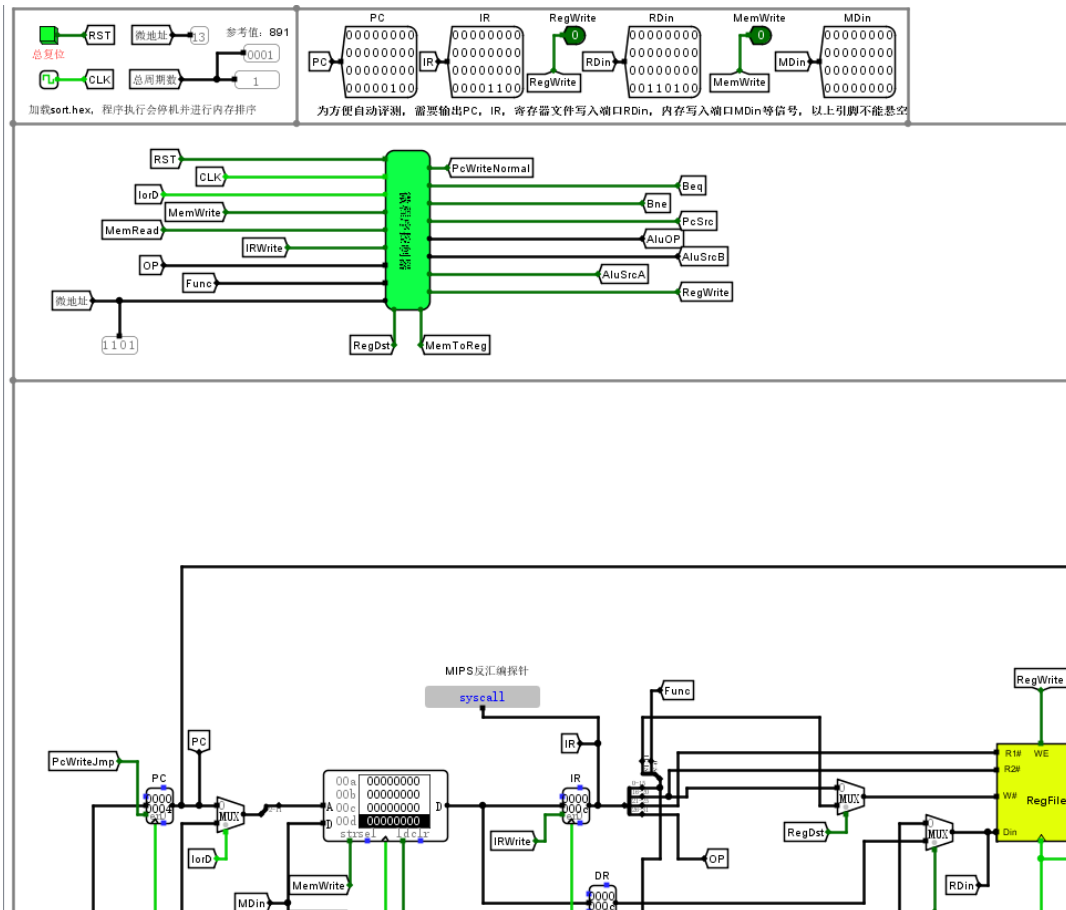
6.6.1 头歌平台测试结果：

3	MIPS微程序CPU设计	20	20.00	已通过	20秒	3	困难	55.21%	微程序CPU设计
---	--------------	----	-------	-----	-----	---	----	--------	----------

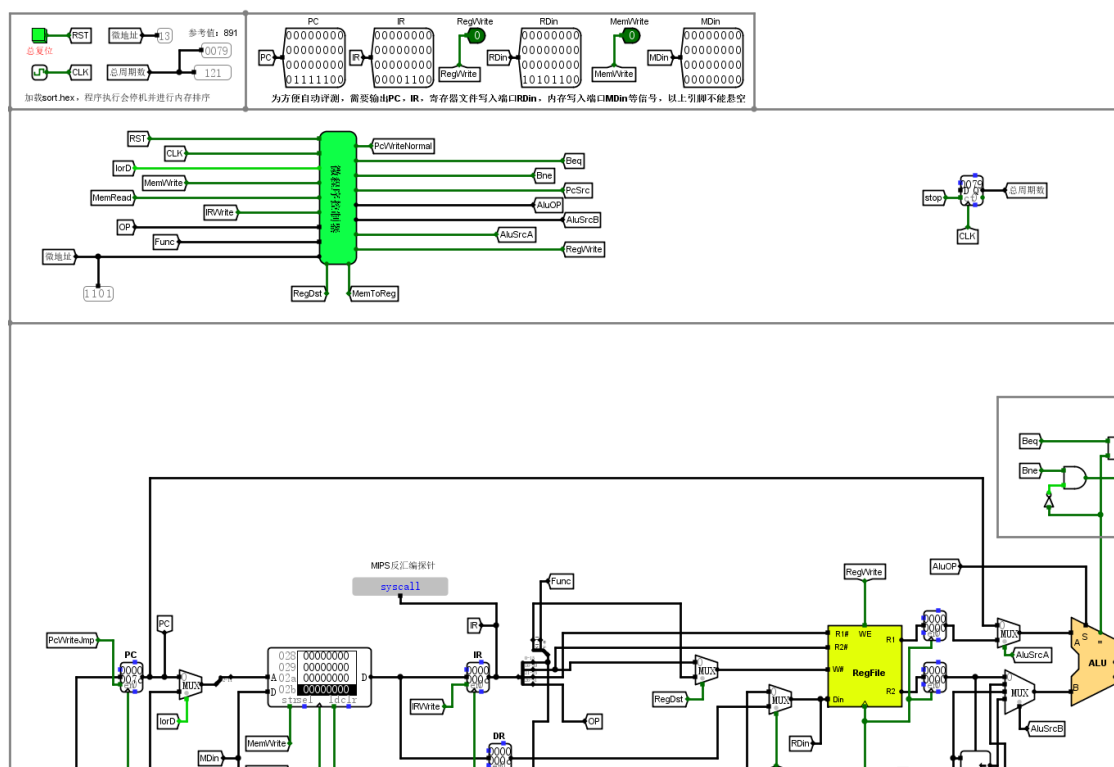
6.6.2 八条指令单独测试用例的测试结果

每条指令都是从标准 Sort.hex 程序中选取的相关指令。优先测试 Syscall 停机指令，并在其他所有测试程序后都添加 Syscall 指令，以保证测试后停机以供检查。

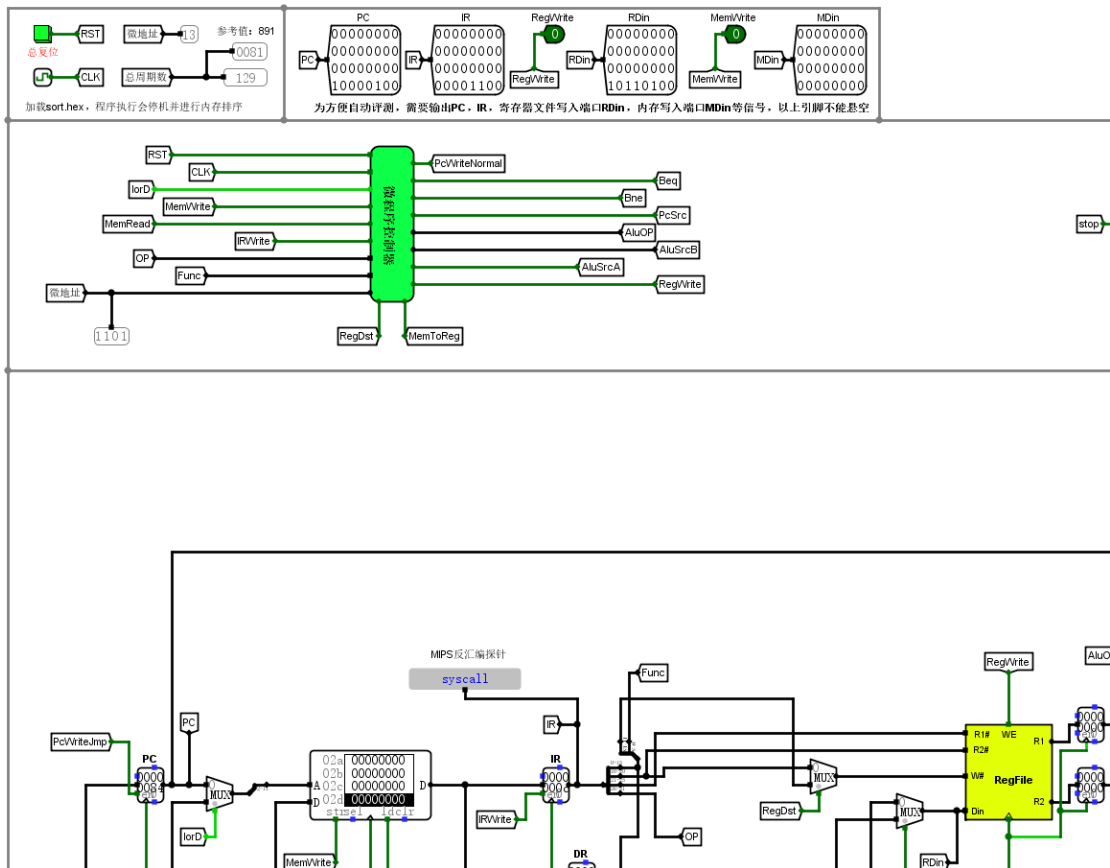
1. SysCall:



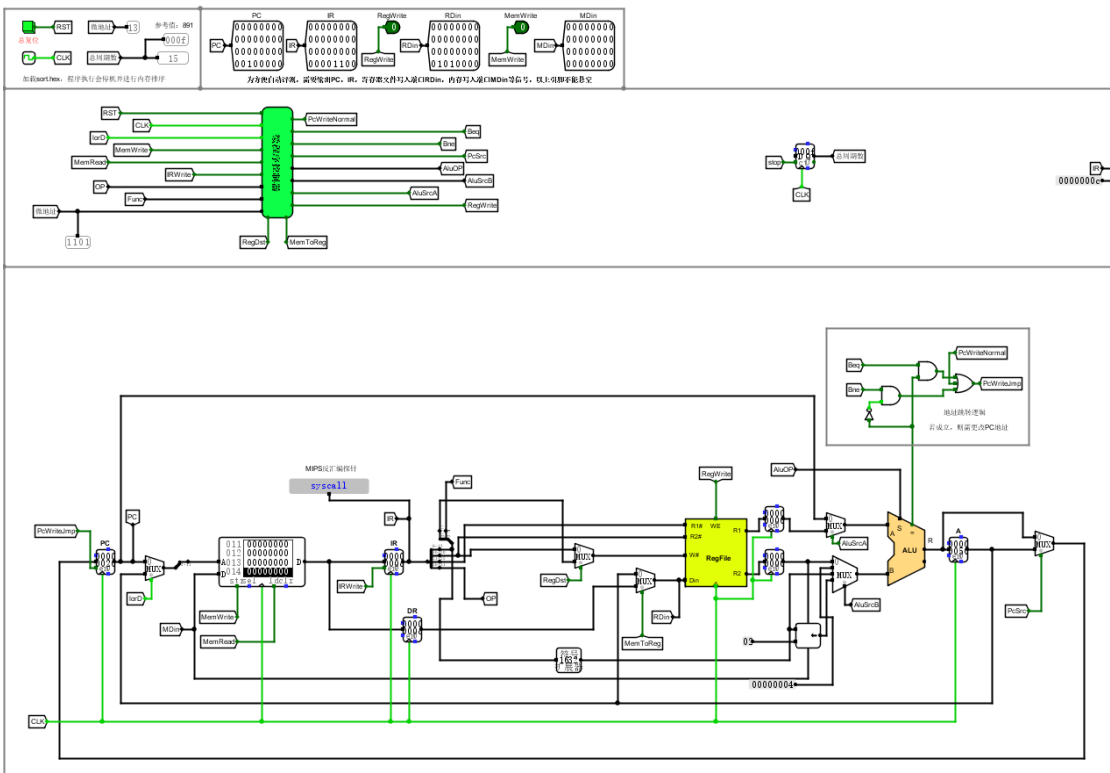
2. Addi



3. Add



4. Beq



5. Bne

实验心得

计算机组成原理课程的所有资料（除课本）都会上传到 Github（后续应该不会维护更新）：

https://github.com/Spercent521/GraduateWorkArchive_CSer

1. 学会了使用工程化方法生成电路，了解了 logisim 常用 debug 方法；
2. 深入理解 Cache 设计机制，了解接口和模块设计机制；
3. 宏观上掌握了 CPU 的运行机理和模块划分，而且还学会了从微观上准确地生成每一个控制信号；
4. 熟悉了 MIPS 核心指令，对 MIPS 的三类指令有了较为大概的实现思路；
5. 实验最大的感受是对于课程内容理解的加深。在上完课以后往往没有真的学懂知识，在做实验的过程中，自然就会主动地复习上课讲过的内容，而且必须真的弄懂才能完成实验；
6. 实验里有一点不好的体验就是文档更新不及时，常常不知道实验需求，建议老师在就实验过程中同学反馈的关键点在课程学习群中发布公告，或者以文档形式实时更新，供其他同学参考；
7. 课设报告模板不合理，太多重复之处，不能体现工作，负担大于收获。下面左图是华科的报告模板目录，右图是本文的原始模板（只截取 CPU 实验）。

▲ 1 CPU设计实验
1.1 设计要求
▷ 1.2 方案设计
▷ 1.3 实验步骤
▷ 1.4 故障与调试
▷ 1.5 测试与分析
▷ 2 总结与心得
参考文献

▲ 实验四、单周期MIPS CPU设计
1、实验目的
2、主要任务
1 实验方案的总体设计
2 实验的实现过程
3 电路图及相关说明
4 实验数据测试
▲ 实验五、微程序地址转移逻辑设计
1、实验目的
2、主要任务
3、实验方案的总体设计
4、实验的实现过程
5、电路图及相关说明
6、实验数据测试
▲ 实验六、MIPS微程序CPU设计
1、实验目的
2、主要任务
3、实验方案的总体设计
4、实验的实现过程
5、电路图及相关说明
6、实验数据测试
实验心得